“Campus Skill Exchange & Micro-Learning Platform”

A Web-Based Platform for Peer Skill Exchange, Micro-Learning and Learning Progress Management

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Course: Database Systems Engineering and Distributed Backend Development (25CS1302E)
Y25 – 2026-2027 | [Trimester Information Required] | Section 1 | Team 12

By

GUTTIKONDA GAYATHRI	**: 2520030465**
G. PAVANI	**: 2520030046**
M. GEETHIKA	**: 2520030056**

Under the Esteemed Guidance of

Dr. Purushotham Muniganti
[Guide Designation Required]
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Declaration

The Project Report entitled **"Campus Skill Exchange & Micro-Learning Platform"** is a record of Bonafide work of **GUTTIKONDA GAYATHRI – 2520030465, G. PAVANI – 2520030046, M. GEETHIKA – 2520030056** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L (Deemed to be) University as part of the course Database Systems Engineering and Distributed Backend Development (25CS1302E). The results embodied in this report have not been copied from any other departments/University/Institute.

GUTTIKONDA GAYATHRI – 2520030465

G. PAVANI – 2520030046

M. GEETHIKA – 2520030056

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report entitled **"Campus Skill Exchange & Micro-Learning Platform"** is a bonafide work done and submitted by **GUTTIKONDA GAYATHRI – 2520030465, G. PAVANI – 2520030046, M. GEETHIKA – 2520030056** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in the Department of Computer Science and Engineering, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in completing this project.

Our greatest appreciation goes to our guide, **Dr. Purushotham Muniganti**, Department of Computer Science and Engineering, whose support, encouragement and guidance throughout the project cannot be expressed in words. We also thank our Course Coordinator for the course *Database Systems Engineering and Distributed Backend Development (25CS1302E)* for the direction and feedback given during the reviews.

We express our gratitude to the Head of the Department of Computer Science and Engineering for providing us with adequate facilities, ways and means by which we were able to complete this project-based course.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

GUTTIKONDA GAYATHRI – 2520030465

G. PAVANI – 2520030046

M. GEETHIKA – 2520030056

Abstract

The Campus Skill Exchange & Micro-Learning Platform is a web-based application that helps students of a college campus teach, learn and track their progress in one place. In most campuses, peer learning depends on informal conversations and scattered websites: students find it difficult to discover who can teach a particular skill, to find a classmate whose learning needs complement their own strengths, and to keep a record of what was learned. This project addresses these problems by providing a centralised platform in which every student maintains a profile, lists the skills he or she can teach and the skills he or she wants to learn, and discovers other students with complementary skills.

The system provides registration and login with JWT-based authentication, profile management, a campus skill library, complementary skill matching (reciprocal and one-way), structured learning requests that move through the states pending, accepted, rejected and completed, a micro-learning course catalogue with enrollment and lesson-based progress tracking, learning-session logging, and reviews with 1–5 star ratings that are allowed only after a learning exchange has been completed. A role flag on the student record enables an administrator module for platform overview, skill creation and course management.

The front end is built with React and Vite, the back end with Python and FastAPI exposing REST APIs, and data is stored in a MySQL database named campus_skill_exchange accessed through the SQLAlchemy ORM with the PyMySQL driver. Passwords are stored as bcrypt hashes and protected endpoints are guarded by signed JWT access tokens. The current version is configured for local development and evaluation; the report documents the design, implementation, API-level testing performed against a running instance, and the limitations and future work of the platform.

Keywords: Peer Learning, Skill Exchange, Skill Matching, Micro-Learning, React.js, Vite, FastAPI, SQLAlchemy, MySQL, JWT Authentication, REST API.

Index

S. No	Chapters	Topics	Page no.
		Acknowledgement Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience 1.7 Team organisation and roles	7-12
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	13-15
3	Technology Stack	3.1 Front-end (React.js) 3.2 Back-end (Python, FastAPI) 3.3 Database (MySQL) 3.4 API communication and ORM (REST API, SQLAlchemy) 3.5 Notification and external services 3.6 Version control (Git, GitHub) 3.7 Development environment (VS Code)	16-19
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (cloud, local, CI/CD pipelines)	20-23
5	Design	5.1 Use case diagram and actors 5.2 ER diagram / Database schema 5.3 Data dictionary 5.4 Learning exchange life-cycle (state model) 5.5 Skill matching algorithm	24-34
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	35-43
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	44-47
8	Testing	8.1 API testing (Swagger UI and scripted requests) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	48-53
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	54-56
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	57-60
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	61-62
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	63-65
13	References	Books, standards, official documentation, tutorials	66-67
14	Appendices	A. Screenshots of the app B. Sample code snippets C. Installation/setup instructions D. User manual or guide E. Geo-tag photos with guide F. Review forms with guide signatures	68-81

CHAPTER - 1 INTRODUCTION

1.1 Background of the Project

Learning in a college does not happen only in classrooms. Students pick up programming languages, design tools, data-analysis techniques, public-speaking habits and many other practical skills from one another, through project teams, clubs, hostel discussions and informal help sessions. Almost every campus contains students who are good at something and students who want to learn exactly that skill. The difficulty is that these two groups rarely find each other efficiently.

At present, peer learning on most campuses is organised through personal contacts, messaging groups and social media communities. A student who wants to learn SQL must know whom to ask; a student who is confident in SQL has no convenient way to make that fact visible to others. Learning material is spread over unrelated websites, and once a help session is over there is usually no record of it, no way to measure progress and no way to give the helper feedback.

The **Campus Skill Exchange & Micro-Learning Platform** developed in this project addresses these needs with a modern full-stack architecture. The front end is a React single-page application built with Vite. The back end is a FastAPI service written in Python that exposes REST APIs, and persistent data is kept in a MySQL database accessed through the SQLAlchemy ORM. Every student can register, describe the skills he or she can teach and wants to learn, and be matched with other students whose skills complement his or her own. Matched students exchange structured learning requests, and the platform additionally offers short micro-learning courses, learning-session logs, progress tracking and peer reviews.

From an academic and engineering perspective, the background of this project rests on four areas of computer science that are central to the course *Database Systems Engineering and Distributed Backend Development*:

1. **Database management** — a relational schema of nine tables for students, skills, student-skill assignments, learning requests, courses, enrollments, sessions, reviews and matches, with primary keys, foreign keys, unique constraints and check constraints that protect the integrity of the data.
2. **Backend and API engineering** — a layered FastAPI service with routers, Pydantic request schemas, dependency injection for database sessions and authentication, and an ORM layer that maps Python classes to MySQL tables.

3. **Authentication and authorisation** — bcrypt password hashing, signed JSON Web Tokens, ownership checks (a student may edit only his or her own data) and an administrator role flag.
4. **Web front-end development** — a component-based single-page application with client-side routing, protected routes, a shared API client and search/filter interfaces.

The remainder of this report documents the requirements, technology choices, architecture, database design, implementation, features, testing, deployment, limitations and future scope of the platform.

Comparison with the Existing Approach

Peer learning on campuses commonly depends on informal communication. The table below summarises how the proposed platform differs from that common approach. It does not claim that any particular institution lacks or has a specific official system.

Aspect	Existing / Traditional Approach	Proposed Platform
Skill discovery	Informal communication and scattered platforms; skills of other students are not visible.	Centralised skill library; each student profile lists teaching and learning skills.
Peer learning	Limited structured opportunities to identify suitable peers.	Direct peer learning through complementary skill matching (reciprocal and one-way matches).
Skill exchange	Coordination through informal chat or social methods; no status or record.	Structured learning requests with tracked status: pending, accepted, rejected, completed.
Learning resources	Learning resources distributed across unrelated platforms.	Integrated micro-learning course catalogue with enrollment.
Progress tracking	No common record of lessons completed or time spent learning.	Lesson-based course progress and logged learning sessions with duration.
Feedback	Feedback, if any, is verbal and unrecorded.	Star ratings and written reviews, allowed only after a completed exchange.
Campus learning	No unified campus-level view of skills, peers, courses and progress.	Single platform integrating skills, peers, courses and progress.

1.2 Problem Statement

Students on a campus possess a wide variety of skills, but there is no convenient, structured way for them to share these skills with one another. Learning needs remain invisible, teaching ability remains undiscovered, and the help that does take place is unrecorded.

The first difficulty is **skill discovery**. A student who wants to learn a skill has no directory to consult. The only option is to ask around, which works only within a small circle of acquaintances and leaves students outside that circle without help.

The second difficulty is **finding a complementary peer**. A productive peer exchange is one in which both students gain: one wants to learn what the other can teach. Identifying such pairs by hand across hundreds of students is impractical, and a one-sided arrangement is often hard to sustain.

The third difficulty is **lack of structure**. Requests for help are made through chat messages that have no status. It is not clear whether a request was seen, accepted, declined or completed, and nothing links a finished session to a review or a record of learning.

The fourth difficulty is **scattered learning resources and progress**. Short courses and practice material exist on many unrelated websites; students cannot see in one place what they have enrolled in, how many lessons they have finished or how much time they have spent learning.

The problem addressed by this project can therefore be stated as follows: *to design and develop a secure, database-backed web platform that lets students of a campus publish the skills they can teach and want to learn, discovers complementary peers automatically, manages learning requests through a defined life-cycle, offers micro-learning courses with progress tracking, and collects reviews after completed exchanges.*

1.3 Scope of the Project

The scope of the project covers the design, development, testing and documentation of a campus-level peer-learning platform. The following functions are within the scope of the present version, and each of them has been verified in the source code of the repository:

- Student registration, login, logout and profile management (name, e-mail, course/branch, year and bio) with JWT-based authentication.
- A campus skill library that guests can browse and search, and in which signed-in students mark skills as ones they can teach or want to learn.
- Complementary skill matching that identifies students who can teach what the logged-in student wants to learn, classified as reciprocal or one-way.
- Learning requests: sending a request to a matched student, viewing sent and received requests, and accepting, rejecting and completing requests.
- A micro-learning course catalogue with filters, enrollment (duplicate enrollment is prevented) and lesson-based progress tracking.
- Logging of learning sessions with duration, date and notes, and aggregated learning hours on the progress page.

- Reviews with 1–5 star ratings after a completed learning exchange, and an average-rating summary on the profile and dashboard.
- An administrator role that can view platform overview figures and all learning requests, list students, add skills to the library and create or edit courses.
- A MySQL database schema with SQL scripts, a demo-data seeder and a REST API documented automatically through Swagger UI.

The following are deliberately outside the scope of the present version: real-time notifications and chat between students, calendar-based scheduling, payments, a native mobile application, e-mail or SMS delivery, institutional single sign-on, AI-based recommendations, user moderation or deletion tools, and cloud deployment. Some of these are listed as future enhancements in Chapter 11.

1.4 Objectives of the Project

The overall objective is to build a reliable, easy-to-use campus platform that connects students who can teach with students who want to learn and records the resulting learning activity. The specific objectives are:

- To design a normalised MySQL schema that stores students, skills, skill assignments, requests, courses, enrollments, sessions and reviews with appropriate constraints.
- To implement secure registration and login using bcrypt password hashing and JWT access tokens, and to protect private data by ownership and role checks.
- To implement a skill library in which each student can record the skills he or she can teach and wants to learn.
- To implement a matching algorithm that finds complementary peers from the stored skill assignments, excluding the logged-in student from his or her own results.
- To implement a learning-request workflow with a defined life-cycle and rules that prevent invalid transitions and duplicate active requests.
- To provide a micro-learning course catalogue with duplicate-safe enrollment and lesson-based progress calculation.
- To record learning sessions and show cumulative learning hours and course progress.
- To collect reviews and ratings only for completed exchanges and to summarise the average rating received by each student.
- To provide an administrator interface for monitoring platform data and managing the skill library and courses.

- To apply database-engineering concepts — keys, constraints, indexes, joins, aggregation and transactions — in a working full-stack application.

1.5 Importance of the Application

Peer learning is one of the most effective and least expensive ways of spreading practical skills within an institution. A student who explains a concept to a classmate understands it better, and a student who receives help from a peer often finds it easier to ask questions than in a classroom. A platform that makes these exchanges easy to find and easy to organise increases the amount of learning that actually takes place.

The application is also significant from a technical and educational point of view. It is a compact but realistic full-stack system that demonstrates how a relational database, a REST API, token-based authentication and a single-page application cooperate. Each module of the platform maps to concepts studied in the database and backend course: entity relationships, many-to-many relationships through a junction table, referential integrity, aggregation queries and role-based access.

Finally, the platform improves transparency. Every request has a visible status, every completed exchange can be reviewed once, and every student can see his or her own learning hours and course progress. This makes campus peer learning visible and measurable instead of informal and invisible.

- Makes teaching ability and learning needs visible across the campus.
- Automatically suggests complementary peers instead of relying on personal contacts.
- Gives every peer-learning exchange a clear status and a review once it is complete.
- Brings courses, sessions and progress together in one place.
- Demonstrates database design, REST APIs and token-based security in a real application.

1.6 Target Users / Audience

The platform serves three kinds of users. The role system is simple: guests are not logged in, students are registered users, and administrators are students whose record has the flag is_admin set.

User Type	Description	Main Activities
Guest	A visitor who has not logged in.	View the home page and public statistics, browse and search the skill library, browse the course catalogue, register and log in.
Student	A registered user of the campus platform.	Maintain the profile, add and remove teaching and learning skills, view matches, send, accept, reject and complete learning requests, enroll in courses, update progress, log sessions and submit reviews.
Administrator	A student account with the administrator flag.	View platform overview figures and all learning requests, list students, add skills to the library, and create or edit courses.

Secondary audiences include faculty members who may use the platform to understand which skills are available among students, and future students of the course who can use the project as a reference implementation of a database-driven web application.

1.7 Team Organisation and Roles

The project was carried out by Team 12 of Section 1 under the guidance of Dr. Purushotham Muniganti, Department of Computer Science and Engineering, as part of the course Database Systems Engineering and Distributed Backend Development (25CS1302E) during the academic year 2026–2027. The team members and their responsibilities are listed below.

Roll Number	Name of the Student	Role / Responsibility
2520030465	GUTTIKONDA GAYATHRI	[Information Required: Confirm each team member's role/responsibility]
2520030046	G. PAVANI	[Information Required: Confirm each team member's role/responsibility]
2520030056	M. GEETHIKA	[Information Required: Confirm each team member's role/responsibility]

The source code of the project is maintained in a GitHub repository (https://github.com/Ggayathri1730/DBMS-Campus-Skill-Exchange-Micro-Learning-Platform).

CHAPTER - 2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Hardware requirements define the physical resources needed to develop, test, run and use the application. The current version runs entirely on a developer machine, so the requirements below describe a development and evaluation setup rather than a production data centre.

Development machine

Component	Minimum	Recommended
Processor	Dual-core, 2.0 GHz (Intel Core i3 / AMD Ryzen 3 class)	Quad-core (Intel Core i5 / AMD Ryzen 5 class or higher)
Memory (RAM)	4 GB	8 GB or more
Storage	5 GB free (tools, dependencies and database)	10 GB or more free, SSD
Display	1366 × 768	1920 × 1080
Network	Required once to download Python and npm packages	Broadband for package installation and documentation
Peripherals	Keyboard and pointing device	External monitor for side-by-side code and browser

Server (hosting) machine

In the verified current implementation the server components — the FastAPI back end, the Vite development server and the MySQL database — run on the same machine as the browser. No separate hosting server is used.

Component	Requirement
Processor and memory	Same as the development machine; the load is a single local user or a small demonstration group.
Storage	A few hundred megabytes for the MySQL data directory and the application files.
Operating system	Windows 10/11 (documented setup), Ubuntu Linux or macOS.
Network ports	Back end 8000, front end 5173 and MySQL 3306 on the local loopback address (127.0.0.1).
Database service	A running local MySQL server with the database campus_skill_exchange.

Client (end-user) device

Any desktop, laptop, tablet or smartphone with a modern web browser (Google Chrome, Microsoft Edge, Mozilla Firefox or Safari) and JavaScript enabled can be used to open the user

interface. The interface uses responsive CSS with breakpoints at 900 px and 560 px so that it adapts to narrower screens.

2.2 Software Requirements

The platform is built entirely from open-source or freely available software. The table below lists the software required to run it; versions are taken from the project files requirements.txt and package.json and from the setup instructions in the repository.

Category	Software	Purpose
Operating system	Windows 10/11, Ubuntu Linux or macOS	Host environment (the repository documents a Windows setup).
Programming language	Python 3.12 or newer	Back-end runtime; virtual environment (venv) and pip for dependencies.
Back-end framework	FastAPI 0.116.1 with Uvicorn 0.35.0	REST API framework and ASGI server.
ORM and driver	SQLAlchemy 2.0.43 and PyMySQL 1.1.1	Object-relational mapping and MySQL connectivity.
Database	MySQL Community Server 8.x (InnoDB)	Relational data store; database name campus_skill_exchange.
Front-end tooling	Node.js runtime and npm	Used only as build tooling for the front end (package installation, Vite dev server and production build); the back end does not run on it.
Front-end framework	React 19, React Router 7, Vite 8	Single-page application, client-side routing and bundling.
Web browser	Chrome, Edge, Firefox or Safari (current versions)	Running the user interface.
Version control	Git and a GitHub account	Source-code management and the shared repository.
Editor / IDE	Visual Studio Code	Development environment.
Database client	MySQL Workbench	Running schema.sql and verify.sql and inspecting tables.
API documentation	Swagger UI (generated by FastAPI at /docs)	Interactive exploration and testing of the REST API.

2.3 Development Tools and Frameworks

Visual Studio Code is the integrated development environment used for editing the Python and React code. MySQL Workbench is used to create the database from schema.sql and to run the verification queries in verify.sql.

Git and GitHub are used for version control. The repository contains the front-end code, the back-end code, the SQL scripts and the project documentation in separate folders (frontend, backend and database).

Python and FastAPI provide the server side. FastAPI uses Python type hints and Pydantic models to validate requests and to generate OpenAPI documentation automatically. Uvicorn serves the application. SQLAlchemy maps Python classes to database tables, and PyMySQL is the driver that talks to MySQL.

React is used to build the user interface from reusable components, React Router handles navigation between pages without full page reloads, and Vite provides a fast development server and the production build. The oxlint tool is used to check the JavaScript code for errors.

The main libraries used are listed below.

Library	Use in the project
fastapi	Web framework: routers, dependency injection, request validation and automatic API documentation.
uvicorn[standard]	ASGI server that runs the FastAPI application.
SQLAlchemy	ORM: model classes, relationships, queries and transactions.
PyMySQL	Pure-Python MySQL driver used through the connection URL mysql+pymysql://.
pydantic[email], email-validator	Request and response schemas; e-mail address validation.
python-jose[cryptography]	Creation and verification of signed JWT access tokens (HS256).
passlib[bcrypt], bcrypt	Password hashing and verification with bcrypt.
python-dotenv	Loads DATABASE_URL, SECRET_KEY and other settings from backend/.env.
python-multipart	Form-data parsing support required by FastAPI.
react, react-dom	Component-based user interface.
react-router-dom	Client-side routing, protected and administrator routes.
vite, @vitejs/plugin-react	Development server and production bundling of the React code.
oxlint	Static analysis (linting) of the front-end source.

CHAPTER - 3 TECHNOLOGY STACK

This chapter describes the technologies chosen for each layer of the Campus Skill Exchange & Micro-Learning Platform and explains why each is suitable. Every technology listed here was verified in the project repository.

Layer / Category	Technology	Role
Front-End Framework	React.js (with HTML, CSS and JavaScript), Vite	Single-page user interface, routing and bundling.
Back-End Language / Framework	Python, FastAPI, Uvicorn	REST API, validation, authentication and business rules.
Database	MySQL (InnoDB)	Durable storage of all application data.
Database Layer	SQLAlchemy ORM, PyMySQL	Mapping classes to tables and executing SQL.
API Communication	REST over HTTP with JSON	Communication between the browser and the server.
Authentication	JWT (python-jose), bcrypt (passlib)	Login tokens and password protection.
Notification / External Services	None integrated	The current version does not use an external notification service.
Version Control	Git, GitHub	Source management and shared repository.
Development Environment	Visual Studio Code, MySQL Workbench, Swagger UI	Editing, database inspection and API exploration.

3.1 Front-End Framework (React.js)

The front end is a single-page application (SPA) written in React with HTML5, CSS3 and JavaScript and bundled by Vite. A single HTML file (index.html) hosts the application; React Router changes the visible page when the user navigates, so the browser does not reload the whole document. This gives a smooth experience while the data shown on each page is always fetched from the back end.

The interface is decomposed into pages and reusable components. The pages are Home, Login and Register (Auth), Dashboard, Profile, Skills, Matching, Requests, Courses, Progress, Reviews and Admin. Shared components include the navigation bar, footer, layout wrappers, protected-route guards, and card components for skills, matches and requests.

The interface uses a dark navy theme written in plain CSS (App.css and index.css). Flexbox and Grid are used for layout, and two media-query breakpoints (900 px and 560 px) adapt the

layout to tablets and phones. The project does not use a UI component library or a global state library; shared authentication state is provided through React Context.

3.2 Back-End (Python, FastAPI)

The back end is built with Python and FastAPI. FastAPI is an asynchronous-capable web framework that uses Python type hints for request validation and generates OpenAPI documentation automatically, which the project uses through the Swagger UI page at /docs. The application is served by Uvicorn.

The server is organised into small, focused modules: main.py creates the application, registers the CORS middleware and includes the routers; database.py configures the SQLAlchemy engine and the per-request database session; models.py defines the ORM classes; schemas.py defines the Pydantic request and response models; auth.py contains password hashing, token creation and the dependencies that identify the current student; and the routers folder contains one router for each functional area (auth, students, skills, matches, requests, courses, progress, reviews, admin and public).

Dependency injection is used heavily. Each endpoint declares the database session and, where needed, the current student (get_current_student) or the administrator guard (require_admin) as function parameters, so security checks are applied uniformly. The back end contains no background workers or schedulers; all processing happens inside request handlers.

3.3 Database (MySQL)

MySQL is the primary database and the single source of truth for all durable data. A peer-learning platform is naturally relational: students have many skills and each skill belongs to many students (a many-to-many relationship resolved by the student_skills junction table), students send requests to each other, enroll in courses, log sessions and review one another. These relationships, together with the need for consistent counts and averages, fit a relational database well.

The schema (database/schema.sql) creates the database campus_skill_exchange with the utf8mb4 character set and nine InnoDB tables: students, skills, student_skills, matches, learning_requests, courses, enrollments, sessions and reviews. InnoDB provides transactions and foreign-key enforcement; every state-changing request in the application ends in a single commit, which is rolled back when an error occurs.

Integrity is enforced in the database as well as in the application: primary keys, UNIQUE constraints (e-mail, student ID, skill name, student–skill pair, student–course pair, one review per reviewer per exchange), foreign keys with ON DELETE CASCADE or SET NULL, CHECK constraints (for example rating between 1 and 5 and requester different from recipient) and an ENUM column for the request status. The rating of a course uses DECIMAL(3,2) and the review rating uses TINYINT.

3.4 API Communication and ORM (REST API, SQLAlchemy)

The browser and the server communicate through a REST API that exchanges JSON over HTTP. Resources are identified by URL paths such as /skills, /matches, /requests and /courses; the HTTP methods GET, POST, PUT, PATCH and DELETE express the action; and standard status codes (200, 201, 204, 400, 401, 403, 404, 409 and 422) report the outcome. Protected endpoints expect the header Authorization: Bearer followed by the JWT. Cross-origin requests from the Vite development server are allowed through FastAPI's CORS middleware, whose allowed origins are read from the CORS_ORIGINS setting.

SQLAlchemy 2.0 is the object-relational mapper. Each table is represented by a Python class that declares columns with Mapped and mapped_column, and relationships declare how tables are linked. A database session is created for each request by the get_db dependency and closed when the request finishes. Queries use the ORM for simple look-ups and joins and SQL functions (count, sum, avg, coalesce) for aggregates, for example the number of students who can teach each skill (an outer join with GROUP BY) or the average rating received by a student.

PyMySQL is the driver used by SQLAlchemy to talk to MySQL. The engine is created with pool_pre_ping enabled so that stale connections are detected before use. The platform does not use WebSockets, message queues or an in-memory cache: every page obtains the current data through ordinary request and response cycles.

3.5 Notification and External Services

The current implementation does not integrate an external notification service. There is no e-mail, SMS or push-notification delivery, no real-time channel and no third-party API. A student sees a change in a request or a review the next time the corresponding page is loaded. Payments are not part of the system.

This is a deliberate simplification for the scope of the course project. Because all business rules live inside the FastAPI back end and the database, a notification service could be added later behind the existing request-status changes without altering the data model (see Chapter 11).

3.6 Version Control (Git, GitHub)

Git is used as the distributed version-control system and GitHub hosts the remote repository at https://github.com/Ggayathri1730/DBMS-Campus-Skill-Exchange-Micro-Learning-Platform. The repository contains three top-level folders — frontend, backend and database — together with the README and testing documents (TEST_CASES.md, TEST_DATA.md, TESTING_INSTRUCTIONS.md, DEMO_GUIDE.md and FINAL_E2E_TEST_REPORT.md).

The commit history records the initial commit (28 August 2026), the implementation of the platform and the merge of the earlier repository history (1 October 2026), and the addition of the final QA report and demonstration documents (2 October 2026). Files containing secrets, such as backend/.env, are excluded from version control by .gitignore, and only template files (.env.example) are committed.

3.7 Development Environment (Visual Studio Code)

Visual Studio Code is the code editor used for development of both the Python back end and the React front end. MySQL Workbench is used to create the database, run schema.sql and inspect the tables with verify.sql. FastAPI's built-in Swagger UI at http://127.0.0.1:8000/docs is used to explore and try the API endpoints.

Dependencies are managed with pip and a Python virtual environment for the back end (requirements.txt) and with npm for the front end (package.json and package-lock.json). The command npm run lint checks the front-end code and npm run build produces the production bundle.

CHAPTER - 4 SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The platform follows a layered client–server architecture. The browser (client layer) runs a React single-page application built with Vite. The application layer is a FastAPI service that exposes REST endpoints and applies authentication, validation and business rules. The data layer is a MySQL database that the service reaches through the SQLAlchemy ORM and the PyMySQL driver. Communication between the browser and the service uses JSON over HTTP, and protected endpoints require a JWT access token in the Authorization header. Figure 4.1 shows the verified architecture.

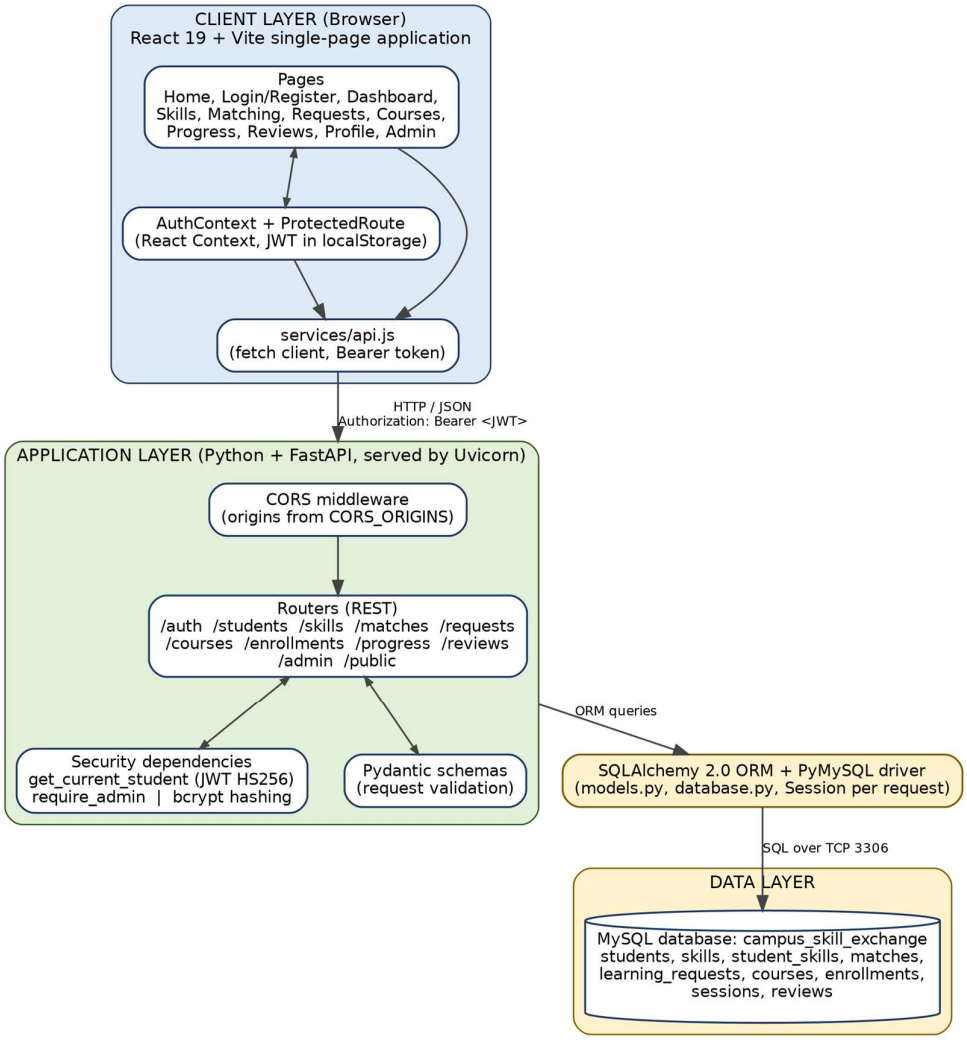


Figure 4.1: High-level architecture of the Campus Skill Exchange & Micro-Learning Platform

4.2 Description of Each Layer (Frontend, Backend, Database)

Client (Frontend) Layer

The client layer is a React single-page application that runs in the user's browser. It presents public pages (home, skills, courses, login and register), student pages (dashboard, profile, matching, requests, progress and reviews) and an administrator page. The module services/api.js wraps the browser fetch function: it adds the base URL, attaches the stored token as a Bearer header, converts error responses into readable messages and returns parsed JSON. The AuthContext keeps the signed-in student and the loading state, and the ProtectedRoute and AdminRoute components redirect visitors who are not allowed to see a page.

Application (Backend) Layer

The application layer is a Python process running FastAPI under Uvicorn. It is organised into the following parts:

- **Routers** — auth, students, skills, matches, requests, courses (with enrollments), progress, reviews, admin and public; each router groups the endpoints of one functional area and is registered in main.py.
- **Security dependencies** — get_current_student decodes the JWT and loads the student; require_admin additionally checks the is_admin flag; passwords are hashed and verified with bcrypt.
- **Schemas (Pydantic)** — define and validate the request bodies and the response shapes, for example the pattern of a student ID, the length of a password and the range of a rating.
- **Matching logic** — the function build_match_results in the matches router computes reciprocal and one-way matches from the stored skill assignments.
- **ORM and session management** — models.py maps the tables to classes, and the get_db dependency provides one database session per request.
- **CORS middleware** — allows the front-end origin configured in CORS_ORIGINS to call the API from the browser.

Data Layer

The data layer consists of a single MySQL database, campus_skill_exchange, with nine tables. It is the only store of durable data; the application keeps no state between requests other than the signed token held by the client. Aggregates such as skill counts, enrollment counts, learning hours and average ratings are computed with SQL queries rather than stored copies, with the

exception of the column enrollment_count in the courses table, which is incremented when a student enrolls.

External Services

No external service is used in the current implementation. There is no e-mail or SMS provider, payment gateway, cloud storage or third-party API (see Section 3.5).

Example Data Flows

Registration. The student fills in the registration form; the React form calls POST /auth/register; FastAPI validates the body with the RegisterRequest schema; the router checks that neither the e-mail nor the student ID exists; the password is hashed with bcrypt; SQLAlchemy inserts a row into students; MySQL commits it; and the API returns the new student record (without the password) with status 201. The front end then redirects the user to the login page.

Login. The student submits the e-mail or student ID with the password; POST /auth/login finds the student, verifies the password against the stored bcrypt hash and returns a signed access token with the student record; the front end stores the token and the user in localStorage and updates the authentication state; subsequent requests carry the token. After a page refresh the application validates the stored token by calling GET /auth/me.

Add a skill. On the Skills page a signed-in student chooses Teach or Learn and selects a skill card; the page calls POST /students/{id}/skills with the skill ID and the two flags; the endpoint checks that the student is editing his or her own profile, creates or updates the row in student_skills, and the page reloads the student's skill list so that the profile shows the change.

Matching. The Matching page calls GET /matches; the router loads the student's own skill rows and the skill rows of all other students from student_skills (joined with skills and students), computes the intersections in memory and returns the matched students sorted by score; the page renders them as match cards.

Learning request. A student selects a match and sends the request; POST /requests validates that the recipient exists, teaches the skill and has no active duplicate request, then inserts a learning_requests row with status pending. The recipient's Requests page lists it, and PUT or PATCH /requests/{id} changes the status to accepted or rejected; either participant can later mark an accepted request completed.

Security and validation concerns cut across all layers: the client validates input and shows server messages; the application layer validates every request again, checks ownership and roles and never returns password hashes; the database enforces uniqueness, foreign keys and check constraints as a last line of defence.

4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

The current version is configured for local development and evaluation. Cloud deployment and automated CI/CD are not part of the verified current implementation.

During development and demonstration the whole stack runs on one machine: the Vite development server serves the React application on port 5173, Uvicorn serves the FastAPI application on port 8000, and a local MySQL server holds the database on port 3306. The browser loads the page from the Vite server and calls the API on port 8000. Configuration is supplied by two untracked environment files: backend/.env (database URL, signing key, token lifetime and allowed origins) and frontend/.env (the API base URL). Figure 4.2 shows this arrangement.

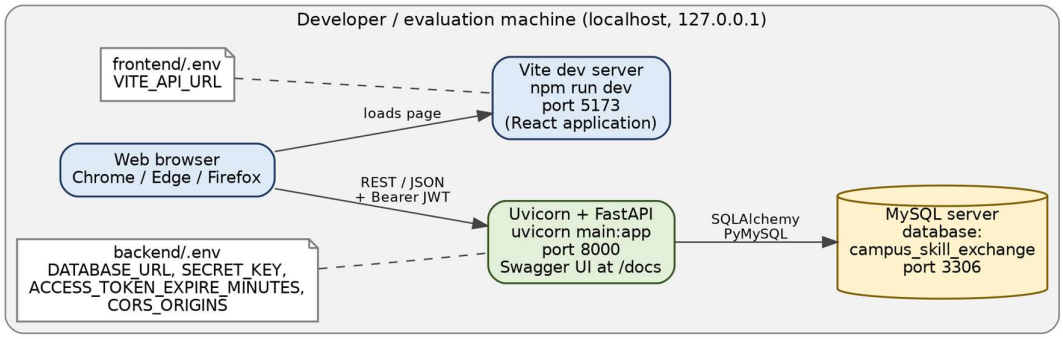


Figure 4.2: Local deployment architecture

No continuous-integration workflow, container configuration or reverse-proxy configuration exists in the repository. Quality checks are run by hand with the commands documented in the repository: the back-end unit test, npm run lint and npm run build. The project's own QA report notes that the application is suitable for local demonstration but would need additional environment hardening and monitoring before any production use.

Scaling. The service is stateless apart from the token carried by the client, so running several Uvicorn workers is a possible future step, but no scaling mechanism is implemented or claimed in this version.

CHAPTER - 5 DESIGN

This chapter presents the design models of the system: the actors and use cases, the entity-relationship diagram and data dictionary of the database, the life-cycle of a learning request and the skill-matching algorithm. All models were derived from the source code and the SQL schema of the repository.

5.1 Use Case Diagram and Actors

The system has three actors. A **Guest** can view the home page and public statistics, browse and search the skill library and the course catalogue, register and log in, but cannot change any data. A **Student** (registered user) can manage the profile and skills, view matches, exchange learning requests, enroll in courses, track progress, log sessions and write reviews. An **Administrator** is a student account whose is_admin flag is set; the administrator can view the platform overview and all learning requests, list students, add skills to the library and create or edit courses. The role system consists of this single flag; there is no separate administrator table, and the registration endpoint cannot set the flag.



Figure 5.1: Use case diagram

5.2 ER Diagram / Database Schema

Figure 5.2 shows the entity-relationship diagram generated from the verified schema. The central entity is **students**. The many-to-many relationship between students and skills is resolved by the junction table **student_skills**, whose two flags can_teach and wants_to_learn record the direction of each assignment. A **learning request** links two students (requester and recipient) and one skill. **Enrollments** form the many-to-many relationship between students and courses and store the progress. **Sessions** record learning activity of a student and may refer to a course, a skill or a learning request. **Reviews** link a reviewer, a reviewed partner and the completed learning request. The table **matches** is defined in the schema but is not written by the current application; matching is computed dynamically (see Section 5.5).

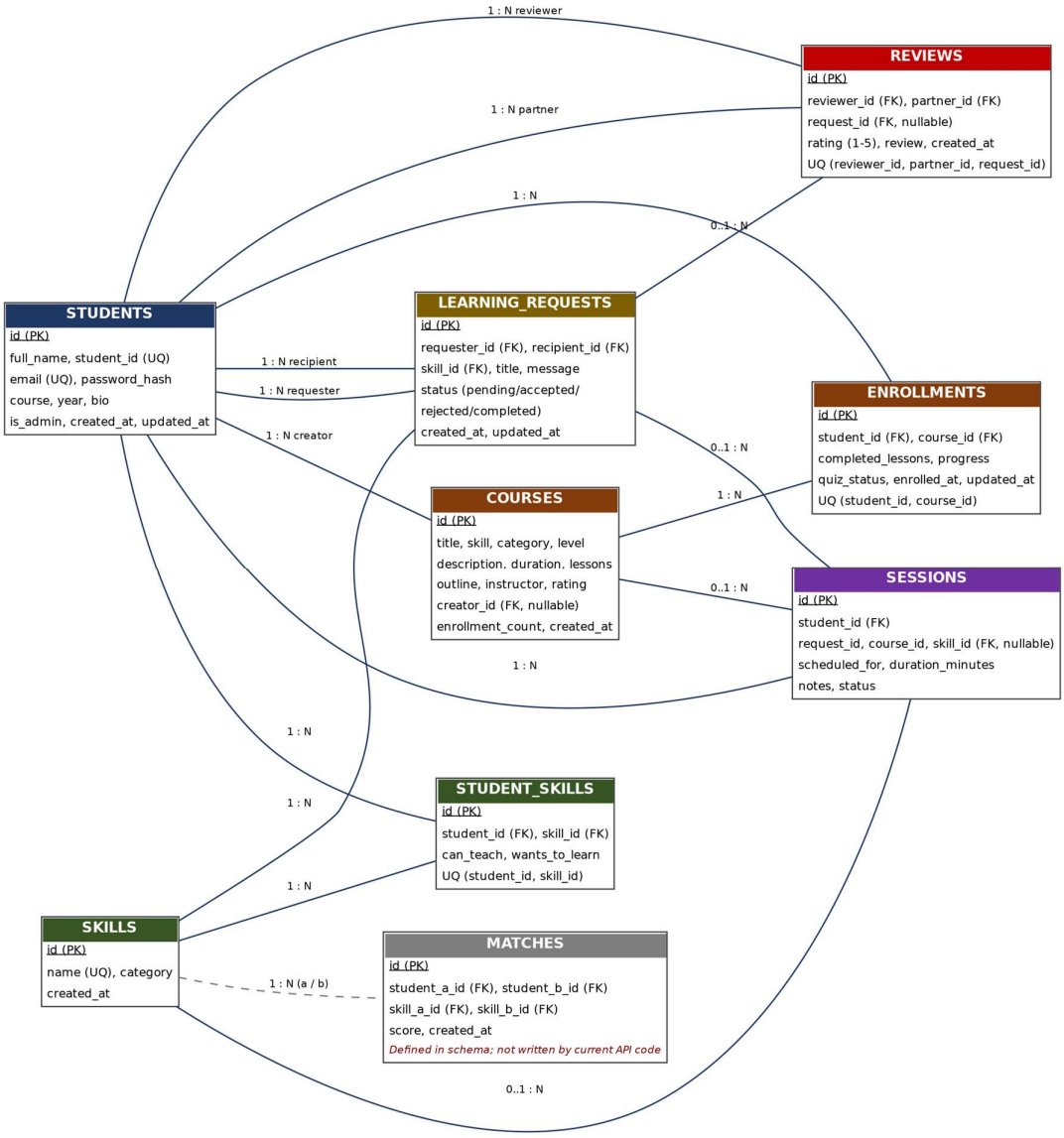


Figure 5.2: Entity-Relationship diagram of the campus_skill_exchange database

Design decisions. (1) The UNIQUE constraint on student_skills(student_id, skill_id) allows one row per student and skill; the direction is stored in two flags so that a skill can be both taught and wanted. (2) A CHECK constraint requires at least one of the two flags to be true. (3) Learning requests carry CHECK requester_id <> recipient_id, so a student cannot send a request to himself or herself. (4) UNIQUE(student_id, course_id) on enrollments guarantees that duplicate enrollment is impossible even if two requests arrive together. (5) UNIQUE(reviewer_id, partner_id, request_id) allows one review per reviewer per exchange. (6) Student-owned rows use ON DELETE CASCADE, whereas references from sessions to courses and skills and from courses to their creator use ON DELETE SET NULL so that history is not lost. (7) The skill of a course is stored as a text name in courses.skill and is not a foreign

key to the skills table. (8) Request status is an ENUM, which restricts the value to the four defined states.

5.3 Data Dictionary

The tables below list every table and attribute of the database with data types and constraints, as defined in database/schema.sql. The same tables are mapped by the SQLAlchemy classes in backend/models.py.

STUDENTS

Purpose: stores registered students and the administrator flag.

Column	Type	Constraints / Description
id	INT	Primary key, auto-increment
full_name	VARCHAR(120)	Not null; display name; indexed
student_id	VARCHAR(40)	Not null, unique; letters, digits and hyphen only (checked by the API)
email	VARCHAR(255)	Not null, unique; validated as an e-mail address by the API
password_hash	VARCHAR(255)	Not null; bcrypt hash, never returned by the API
course	VARCHAR(120)	Not null; course / branch, for example Computer Science
year	VARCHAR(40)	Not null; year of study as text, for example 2nd year
bio	TEXT	Not null; short biography (empty string by default in the application)
is_admin	BOOLEAN	Not null, default FALSE; administrator flag
created_at	DATETIME	Not null, default CURRENT_TIMESTAMP
updated_at	DATETIME	Not null, default CURRENT_TIMESTAMP, updated automatically on change

SKILLS

Purpose: the campus skill library.

Column	Type	Constraints / Description
id	INT	Primary key, auto-increment
name	VARCHAR(120)	Not null, unique; for example Python
category	VARCHAR(80)	Not null; indexed; for example Programming
created_at	DATETIME	Not null, default CURRENT_TIMESTAMP

STUDENT_SKILLS

Purpose: junction table linking students and skills with the direction of each assignment.

Column	Type	Constraints / Description
id	INT	Primary key, auto-increment
student_id	INT	Not null; foreign key to students(id), ON DELETE CASCADE
skill_id	INT	Not null; foreign key to skills(id), ON DELETE CASCADE
can_teach	BOOLEAN	Not null, default FALSE; the student can teach this skill
wants_to_learn	BOOLEAN	Not null, default FALSE; the student wants to learn this skill
(constraints)	—	UNIQUE (student_id, skill_id); CHECK (can_teach OR wants_to_learn)

MATCHES

Purpose: defined for storing matched pairs. The current application does not insert into or read from this table; matches are computed on request.

Column	Type	Constraints / Description
id	INT	Primary key, auto-increment
student_a_id	INT	Not null; foreign key to students(id), ON DELETE CASCADE
student_b_id	INT	Not null; foreign key to students(id), ON DELETE CASCADE
skill_a_id	INT	Not null; foreign key to skills(id)
skill_b_id	INT	Not null; foreign key to skills(id)
score	INT	Not null, default 100
created_at	DATETIME	Not null, default CURRENT_TIMESTAMP
(constraints)	—	UNIQUE (student_a_id, student_b_id, skill_a_id, skill_b_id); CHECK (student_a_id <> student_b_id)

LEARNING_REQUESTS

Purpose: peer-learning requests and their status.

Column	Type	Constraints / Description
id	INT	Primary key, auto-increment
requester_id	INT	Not null; foreign key to students(id), ON DELETE CASCADE
recipient_id	INT	Not null; foreign key to students(id), ON DELETE CASCADE
skill_id	INT	Not null; foreign key to skills(id); the skill the requester wants to learn
title	VARCHAR(160)	Not null, default 'Learning exchange'
status	ENUM	Not null, default 'pending'; values pending, accepted, rejected, completed; indexed
message	TEXT	Not null; description of what the requester wants to learn

Column	Type	Constraints / Description
created_at	DATETIME	Not null, default CURRENT_TIMESTAMP
updated_at	DATETIME	Not null, default CURRENT_TIMESTAMP, updated automatically
(constraints)	—	CHECK (requester_id <> recipient_id); index on (requester_id, recipient_id)

COURSES

Purpose: the micro-learning course catalogue.

Column	Type	Constraints / Description
id	INT	Primary key, auto-increment
title	VARCHAR(160)	Not null
skill	VARCHAR(120)	Not null; name of the related skill (text, not a foreign key); indexed
category	VARCHAR(80)	Not null, default 'General'; indexed
description	TEXT	Not null
level	VARCHAR(40)	Not null; for example Beginner or Intermediate
duration	VARCHAR(40)	Not null; for example 3 weeks
lessons	INT	Not null; CHECK (lessons > 0); used to compute progress
outline	TEXT	Not null; course outline
creator_id	INT	Nullable; foreign key to students(id), ON DELETE SET NULL; indexed
instructor	VARCHAR(120)	Not null, default 'Campus Exchange'
rating	DECIMAL(3,2)	Not null, default 0; no application code currently updates it
enrollment_count	INT	Not null, default 0; incremented when a new enrollment is created
created_at	DATETIME	Not null, default CURRENT_TIMESTAMP

ENROLLMENTS

Purpose: enrollment of a student in a course together with the progress.

Column	Type	Constraints / Description
id	INT	Primary key, auto-increment
student_id	INT	Not null; foreign key to students(id), ON DELETE CASCADE
course_id	INT	Not null; foreign key to courses(id), ON DELETE CASCADE
completed_lessons	INT	Not null, default 0
progress	INT	Not null, default 0; CHECK (progress BETWEEN 0 AND 100)
quiz_status	VARCHAR(40)	Not null, default 'Not started'
enrolled_at	DATETIME	Not null, default CURRENT_TIMESTAMP

Column	Type	Constraints / Description
updated_at	DATETIME	Not null, default CURRENT_TIMESTAMP, updated automatically
(constraints)	—	UNIQUE (student_id, course_id)

SESSIONS

Purpose: learning sessions recorded by students.

Column	Type	Constraints / Description
id	INT	Primary key, auto-increment
request_id	INT	Nullable; foreign key to learning_requests(id), ON DELETE CASCADE
student_id	INT	Not null; foreign key to students(id), ON DELETE CASCADE; indexed
course_id	INT	Nullable; foreign key to courses(id), ON DELETE SET NULL; indexed
skill_id	INT	Nullable; foreign key to skills(id), ON DELETE SET NULL
scheduled_for	DATETIME	Nullable; date and time of the session
duration_minutes	INT	Not null, default 0; CHECK (duration_minutes >= 0)
notes	TEXT	Not null; what was practised
status	VARCHAR(30)	Not null; set to completed by the application when a session is logged

REVIEWS

Purpose: reviews and star ratings given after a completed exchange.

Column	Type	Constraints / Description
id	INT	Primary key, auto-increment
reviewer_id	INT	Not null; foreign key to students(id), ON DELETE CASCADE
partner_id	INT	Not null; foreign key to students(id), ON DELETE CASCADE; indexed
request_id	INT	Nullable; foreign key to learning_requests(id), ON DELETE SET NULL
rating	TINYINT	Not null; CHECK (rating BETWEEN 1 AND 5)
review	TEXT	Not null; written review
created_at	DATETIME	Not null, default CURRENT_TIMESTAMP
(constraints)	—	CHECK (reviewer_id <> partner_id); UNIQUE (reviewer_id, partner_id, request_id)

Indexes

- students(email), students(student_id) and skills(name) — UNIQUE; used for login look-ups and duplicate prevention.
- student_skills(student_id) and student_skills(skill_id) — speed up loading a student's skills and counting teachers per skill.
- learning_requests(status) and learning_requests(requester_id, recipient_id) — support filtering and the duplicate-request check.
- enrollments(student_id) and the UNIQUE pair (student_id, course_id) — list a student's courses and prevent duplicate enrollment.
- sessions(student_id), sessions(course_id), reviews(partner_id), courses(skill), courses(category), skills(category) — support listing and filtering.

Note. The SQL script is the reference definition: it adds ENUM and CHECK constraints that the ORM column types alone do not create. When the application starts it calls create_all, which creates any table that is missing, but the database itself must already exist (created by schema.sql).

5.4 Learning Exchange Life-Cycle (State Model)

Every learning request passes through a defined sequence of states, and only the transitions shown in Figure 5.3 are accepted by the PUT/PATCH /requests/{id} endpoint. The status values are exactly those stored in the database and used in the code: pending, accepted, rejected and completed.

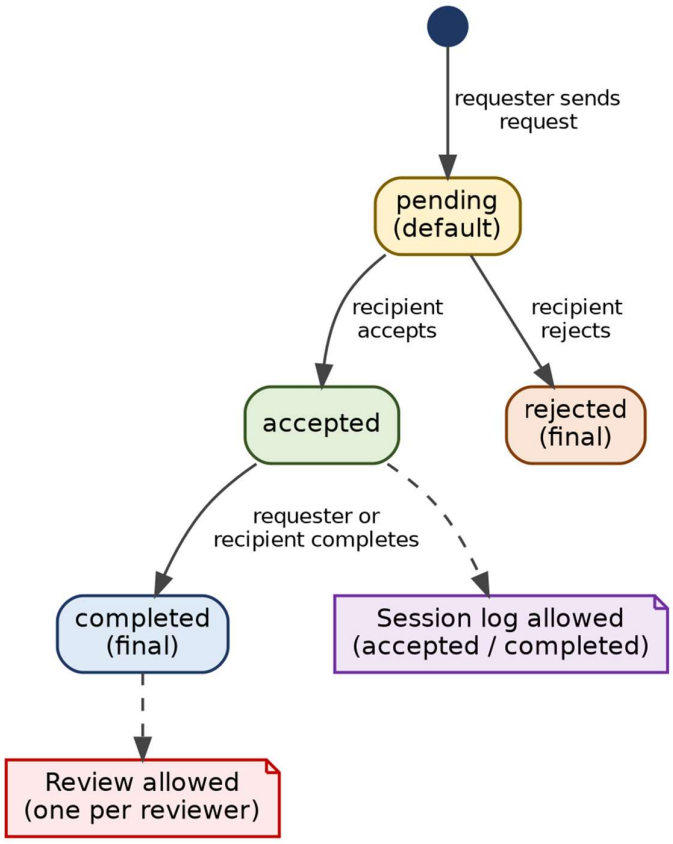


Figure 5.3: Learning exchange (learning request) life-cycle

State	Meaning	Next states
pending	Created by the requester; waiting for the recipient's decision. This is the default status.	accepted or rejected — only by the recipient
accepted	The recipient agreed to teach; the exchange is under way. A learning session can be linked to the request.	completed — by the requester or the recipient
rejected	The recipient declined the request. Final state.	None (a new request may be sent later)
completed	The exchange is finished. Final state; both participants may now submit one review each.	None

Any other change returns HTTP 409 (for example completing a pending request), and an attempt by the requester to accept or reject a pending request returns HTTP 403. Creating a request is also guarded: a second request with the same requester, recipient and skill is refused while an earlier one is pending or accepted.

5.5 Skill Matching Algorithm

Skill matching is the core algorithm of the system. It is implemented by the function build_match_results in backend/routers/matches.py and is executed whenever the Matching,

Requests or Dashboard page asks for matches (GET /matches). Matching is **calculated dynamically** from the rows of student_skills; no result is stored, and no scoring model or machine learning is involved. Figure 5.4 shows the flow.

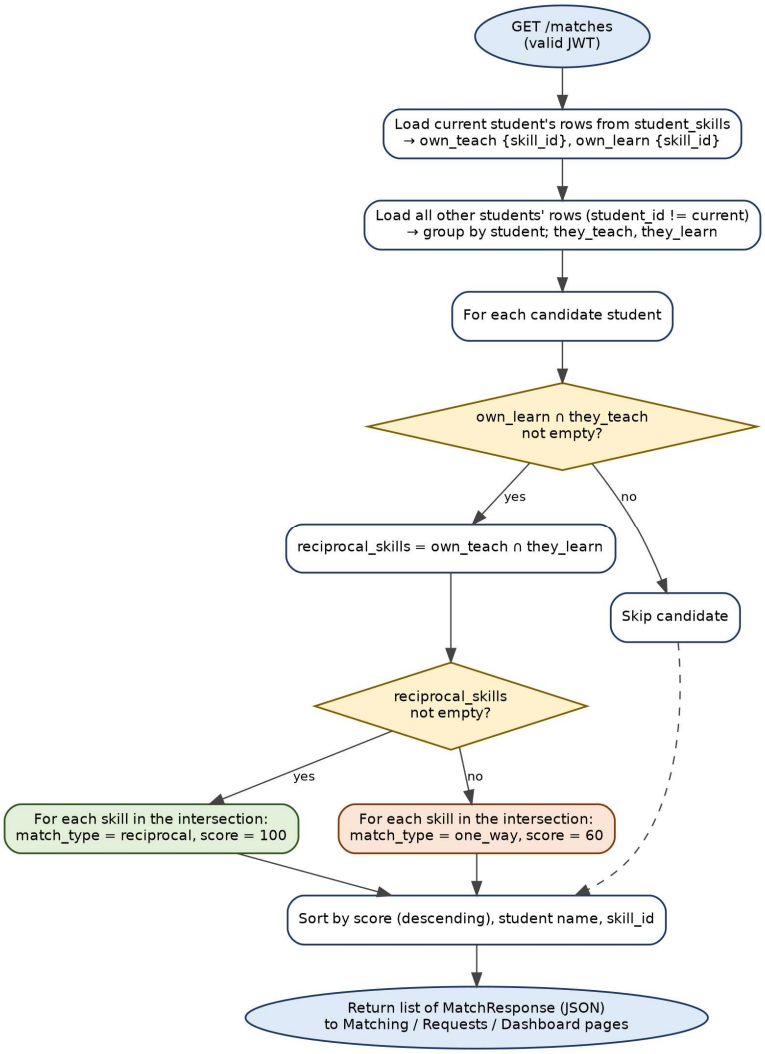


Figure 5.4: Flowchart of the skill matching algorithm

Representation. Each row of student_skills stores a student, a skill and two flags. For the logged-in student the algorithm builds the set own_teach (skills with can_teach) and the set own_learn (skills with wants_to_learn). For every other student it builds they_teach and they_learn in the same way.

Matching rule. Another student is a candidate when he or she can teach at least one skill that the logged-in student wants to learn (own_learn ∩ they_teach). One match record is produced for each such skill, so the same student and skill never appear twice. The match is **reciprocal** when the logged-in student can also teach at least one skill that the candidate wants to learn

(own_teach ∩ they_learn); otherwise it is **one-way**. Reciprocal matches receive the score 100 and one-way matches the score 60, and the results are sorted by descending score, then by student name and skill. The logged-in student is excluded because only rows with a different student ID are examined.

Pseudocode

```
function find_matches(me):
    own_teach = skills where me.can_teach;  own_learn = skills where me.wants_to_learn
    for each other student s (student_id != me):
        they_teach = skills where s.can_teach;  they_learn = skills where
s.wants_to_learn
        desired    = own_learn ∩ they_teach
        reciprocal = own_teach ∩ they_learn
        for each skill in sorted(desired):
            add match(student = s, skill = skill,
                      type  = "reciprocal" if reciprocal is not empty else "one way",
                      score = 100          if reciprocal is not empty else 60)
    return matches sorted by (-score, student name, skill id)
```

Worked example. Suppose the logged-in student can teach Python and wants to learn Java. Student A can teach Java and wants to learn Python: the intersection own_learn ∩ they_teach is {Java} and own_teach ∩ they_learn is {Python}, so the result is a reciprocal match for Java with score 100. Student B can teach Java but does not want to learn anything the logged-in student can teach: the result is a one-way match with score 60. A student who teaches nothing the logged-in student wants to learn produces no match. This scenario is exactly the one checked by the repository's unit test.

Related endpoints. GET /matches/recommended returns the same results sorted by score, and GET /matches/{student_id} returns the matches of a given student but only to that student or to an administrator. **Cost.** The implementation loads all skill rows of other students into memory for each request and then performs set operations per candidate; this is simple and adequate for a campus-sized demonstration but is a point to optimise (for example with a SQL join) if the number of students grows large.

CHAPTER - 6 IMPLEMENTATION

6.1 Module-Wise Implementation

The platform is implemented as a set of cooperating modules with clear responsibilities. Each module has a front-end page or component, one or more REST endpoints, a router in the back end, validation rules and one or more database tables. The modules below were verified by reading the source code and by exercising the endpoints against a running instance (Chapter 8). All endpoints other than those marked public require a valid JWT.

1. User Registration and Authentication Module

Handles account creation, login, logout and the validation of the signed-in user.

Aspect	Details
User interaction	A visitor fills in full name, student ID, college e-mail, password, confirm password, course/branch and year on the Register page and presses Create account; later logs in with the student ID or e-mail and the password; logs out with the Log out button.
Front-end page / component	pages/Auth.jsx (Login and Register), context/AuthContext.jsx, components/Navbar.jsx.
API calls	POST /auth/register, POST /auth/login, GET /auth/me.
Back-end router / service	routers/auth.py; auth.py (hash_password, verify_password, create_access_token, get_current_student).
Validation	Pydantic: name 2–120 characters; student ID 2–40 characters (letters, digits, hyphen); valid e-mail; password 6–128 characters and at most 72 bytes (bcrypt limit); course and year required. The client also checks that the two passwords match. A duplicate e-mail or student ID returns 409; wrong credentials return 401 with one generic message.
Database table	students (password stored only as a bcrypt hash).
Result shown	After registration the user is taken to the login page; after login to the dashboard.
Authentication / authorisation	Registration and login are public. The registration schema does not accept the is_admin field, so an account cannot make itself an administrator.

2. Student Profile Management Module

Lets a student view and update his or her own profile and shows summary figures.

Aspect	Details
User interaction	The student opens the Profile page (from the avatar or the Edit profile link on the dashboard), edits name, e-mail, course, year and bio, and saves.
Front-end page / component	pages/Profile.jsx, pages/Dashboard.jsx.
API calls	GET /students/me/summary; PUT /students/{id} (PATCH is also accepted); GET /students/{id}.
Back-end router / service	routers/students.py.

Aspect	Details
Validation	ProfileUpdate schema (optional fields with the same limits as registration); a changed e-mail that already exists returns 409.
Database table	students; the summary also reads enrollments, sessions and reviews.
Result shown	Updated profile; summary of learning hours, completed courses, average rating and review count.
Authentication / authorisation	JWT required. A student can read or edit only his or her own record; an administrator can access any record (403 otherwise).

3. Skill Management Module

Maintains the campus skill library and each student's teaching and learning skills.

Aspect	Details
User interaction	On the Skills page (My Skills when signed in) the student chooses Teach or Learn, searches or filters by category, and selects a skill card to add it or remove it; the skills in the student's orbit are listed below the cards.
Front-end page / component	pages/Skills.jsx, components/Cards.jsx (SkillCard).
API calls	GET /skills (public); GET, POST /students/{id}/skills; DELETE /students/{id}/skills/{skill_id}; POST /skills (administrator).
Back-end router / service	routers/skills.py.
Validation	The student and skill must exist (404); at least one direction must be chosen (422); adding an existing assignment updates its flags, so no duplicate row is created.
Database tables	skills, student_skills.
Result shown	Skill cards with the numbers of students who can teach and want to learn each skill; the student's own Teach and Learn tags.
Authentication / authorisation	Browsing the library is public. Changing assignments requires the owner or an administrator; creating a new skill requires an administrator.

4. Skill Matching Module

Finds students with complementary skills (Section 5.5).

Aspect	Details
User interaction	The student opens Matching, or the dashboard section People worth meeting, and sees matched students with the skill each can teach; a request can be sent directly from a match card.
Front-end page / component	pages/Matching.jsx, components/Cards.jsx (MatchCard), pages/Dashboard.jsx.
API calls	GET /matches, GET /matches/recommended, GET /matches/{student_id}.
Back-end router / service	routers/matches.py (find_matches, build_match_results).
Validation	The path variant checks that the caller is the same student or an administrator (403) and that the student exists (404).

Aspect	Details
Database tables	student_skills, skills, students (read only).
Result shown	A list of matches with student name, course and year, taught skill, match type (reciprocal or one-way) and score.
Authentication / authorisation	JWT required; the logged-in student is always excluded from his or her own results.

5. Learning Request Module

Manages requests for peer teaching from creation to completion (Section 5.4).

Aspect	Details
User interaction	On the Requests page the student chooses a learning partner from the matches, the skill, a title and a description, and presses Send request; filter tabs (All, Incoming, Sent, Pending, Accepted, Rejected, Completed) list the requests; the recipient accepts or rejects, and a participant completes an accepted request.
Front-end page / component	pages/Requests.jsx, components/Cards.jsx (RequestCard).
API calls	GET /requests; POST /requests; PUT or PATCH /requests/{id}.
Back-end router / service	routers/requests.py.
Validation	Recipient and skill must exist; no request to oneself (400); the recipient must be listed as teaching the skill (422); no second active request for the same requester, recipient and skill (409); only the recipient may accept or reject (403); invalid transitions return 409.
Database table	learning_requests.
Result shown	Request cards with status and date; counts on each filter tab.
Authentication / authorisation	JWT required; a student sees only requests in which he or she is the requester or the recipient.

6. Micro-Learning Course Module

Provides the course catalogue and enrollment.

Aspect	Details
User interaction	The user browses courses, filters by category and level, searches, and presses Enroll now; the tabs Browse all, My enrollments and My courses narrow the list. An administrator sees controls to add and edit courses.
Front-end page / component	pages/Courses.jsx.
API calls	GET /courses and GET /courses/{id} (public); POST /courses/{id}/enroll; GET /enrollments; POST and PUT /courses (administrator).
Back-end router / service	routers/courses.py (router and enrollments_router).
Validation	The course must exist (404). Enrolling again in the same course returns the existing enrollment with status 200 instead of creating a duplicate; the unique constraint uq_enrollment backs this up.

Aspect	Details
Database tables	courses, enrollments.
Result shown	Course cards with level, duration, instructor, enrollment count and an Enroll now button; a confirmation message after enrollment.
Authentication / authorisation	Catalogue is public; enrollment requires a JWT; creating, editing or deleting a course requires an administrator.

7. Learning Sessions and Progress Module

Tracks lesson progress for enrolled courses and records learning sessions.

Aspect	Details
User interaction	On the Progress page the student sees learning hours, completed sessions, active and completed courses and overall progress; advances a course by one lesson; and logs a session by choosing a course or skill, duration, date and notes.
Front-end page / component	pages/Progress.jsx.
API calls	GET /progress; PUT /progress/{enrollment_id}; GET and POST /progress/sessions.
Back-end router / service	routers/progress.py.
Validation	Progress = round(completed lessons ÷ total lessons × 100), with completed lessons capped at the course total. A session needs a course, skill or learning request (422) and a duration of 1–1440 minutes; a linked request must involve the student and be accepted or completed (403/409).
Database tables	enrollments, sessions (also courses, skills and learning_requests for references).
Result shown	Learning hours (sum of completed session minutes ÷ 60), progress ring and list of recent sessions.
Authentication / authorisation	JWT required; every query is restricted to the logged-in student.

8. Reviews and Ratings Module

Collects feedback after a completed exchange.

Aspect	Details
User interaction	The student opens Reviews, selects a completed exchange, chooses a rating from 1 to 5 stars, writes a review and presses Submit review; the page lists received and given reviews with an average rating.
Front-end page / component	pages/Reviews.jsx.
API calls	GET /reviews; GET /reviews/{student_id}; POST /reviews.
Back-end router / service	routers/reviews.py.
Validation	The exchange must exist (404) and have status completed (409); the reviewer must have taken part (403); the partner must be the other participant (422); a reviewer can review an exchange only once (409); rating 1–5; text 2–2000 characters.
Database table	reviews (the average rating is computed from it with AVG).

Aspect	Details
Result shown	Review list, counts of received and given reviews and the average rating; the profile and dashboard show the same average.
Authentication / authorisation	JWT required.

9. Administrator Module

Gives administrators an overview of the platform and basic management functions.

Aspect	Details
User interaction	An administrator opens the Admin page from the navigation bar to see platform totals, the list of students and learning requests, and a form to add a skill; course creation and editing are available on the Courses page.
Front-end page / component	pages/Admin.jsx; AdminRoute in components/ProtectedRoute.jsx.
API calls	GET /admin/overview; GET /admin/requests; GET /students; POST /skills; POST, PUT, DELETE /courses.
Back-end router / service	routers/admin.py; the require_admin dependency in auth.py.
Validation	Duplicate skill names return 409; course data is validated by the CourseCreate schema (lessons at least 1).
Database tables	students, skills, learning_requests, courses, reviews.
Result shown	Totals of students, skills, active requests, courses and reviews; request and student tables; confirmation messages.
Authentication / authorisation	JWT and is_admin = true are required; other users receive 403 from the API and are redirected to the dashboard by the front end. The module is partial: there are no deletion or moderation functions for students, skills or courses in the user interface (a course-delete endpoint exists in the API only).

6.2 Front-End Logic (UI Rendering and State Management)

The React application is organised into pages (Home, Login, Register, Dashboard, Profile, Skills, Matching, Requests, Courses, Progress, Reviews and Admin) and shared components (Navbar, Footer, Layout wrappers, Cards, AsyncState and the route guards). Routing is defined in App.jsx with React Router. Two layouts are used: PublicLayout for pages visible to everybody and AppLayout for the signed-in area.

Route	Page	Access
/	Home (public statistics, featured skills and courses)	Public
/login, /register	Login and Register	Public
/skills, /courses	Skill library, course catalogue	Public (extra actions need login)

Route	Page	Access
/dashboard	Dashboard	Signed-in student
/profile	Profile	Signed-in student
/matching, /requests	Matching, Learning requests	Signed-in student
/progress, /reviews	Progress and sessions, Reviews	Signed-in student
/admin	Administration	Administrator only
any other path	Redirected to /	—

State management. Three kinds of state are used. *Global state* — the signed-in user and the loading flag — lives in AuthContext (React Context) and is saved in the browser's localStorage together with the token. *Page state* — lists loaded from the API, form values, selected tabs, messages and errors — is kept with the useState hook in each page. *Derived state* — filtered lists, counts, averages and percentages — is computed during rendering from the loaded data, for example the filtered skills on the Skills page and the average rating on the Reviews page.

Data loading. Pages load data in useEffect using the functions of services/api.js, often with Promise.all to request several resources in parallel (the dashboard loads skills, matches, requests, progress, reviews and the summary together). A flag prevents state updates after a page has been left, and loading and error messages are displayed while a request is pending or when it fails.

Forms and validation. Forms use controlled inputs. The registration form checks that the password and its confirmation match; other rules (lengths, formats and business rules) are enforced by the server and its messages are shown to the user. Submit buttons are disabled while a request is being saved.

Search and filters. The skill library offers a search box, category buttons and I can teach / I want to learn toggles; the course catalogue offers a search box, category and level filters and the tabs Browse all, My enrollments and My courses; the Requests and Reviews pages offer status tabs. All of them filter the already loaded lists in the browser.

Responsive design. The CSS uses Flexbox and Grid and two media-query breakpoints (900 px and 560 px) to rearrange the layout on smaller screens.

6.3 API Integration

The client communicates with the server through a REST API that returns JSON. The base URL is read from the environment variable VITE_API_URL (default http://127.0.0.1:8000).

All calls pass through one helper function in services/api.js that adds the Authorization header when a token is stored and converts failures into error messages; the object api exposes one function per endpoint.

REST endpoints

Method & Path	Access	Description
GET /	Public	Health check; confirms that the database is connected
GET /public/stats	Public	Totals and featured skills and courses for the home page
POST /auth/register	Public	Create a student account (201)
POST /auth/login	Public	Verify credentials and return a JWT with the student
GET /auth/me	Student	Return the student identified by the token
GET /students/me/summary	Student	Profile with skills, learning hours, completed courses and average rating
GET /students	Administrator	List all students
GET /students/{id}	Self or administrator	Read a student record
PUT, PATCH /students/{id}	Self or administrator	Update the profile
GET /skills	Public	List skills with teacher and learner counts
POST /skills	Administrator	Add a skill to the library
GET /students/{id}/skills	Student	List a student's skill assignments
POST /students/{id}/skills	Self or administrator	Add or update a teach/learn assignment
DELETE /students/{id}/skills/{skill_id}	Self or administrator	Remove an assignment (204)
GET /matches	Student	Matches of the logged-in student
GET /matches/recommended	Student	Same matches sorted by score
GET /matches/{student_id}	Self or administrator	Matches of the given student
GET /requests	Student	Requests sent or received by the student
POST /requests	Student	Send a learning request (201)
PUT, PATCH /requests/{id}	Participant	Accept, reject or complete a request
GET /courses, GET /courses/{id}	Public	List or read courses
POST /courses	Administrator	Create a course (201)
PUT /courses/{id}	Administrator	Update a course
DELETE /courses/{id}	Administrator	Delete a course (API only)
POST /courses/{id}/enroll	Student	Enroll in a course (201, or 200 if already enrolled)
GET /enrollments	Student	Courses the student is enrolled in
GET /progress	Student	Progress of each enrollment
PUT /progress/{enrollment_id}	Student	Update completed lessons and quiz status
GET, POST /progress/sessions	Student	List or log learning sessions
GET /reviews	Student	Reviews given or received by the student
GET /reviews/{student_id}	Student	Reviews received by a given student

Method & Path	Access	Description
POST /reviews	Student	Submit a review for a completed exchange (201)
GET /admin/overview	Administrator	Totals for the platform
GET /admin/requests	Administrator	All learning requests

Request validation (Pydantic schemas)

Every request body is declared as a Pydantic model in schemas.py. FastAPI validates the body before the handler runs and returns status 422 with a description of the problem when a field is invalid. The most important rules are: RegisterRequest and LoginRequest (password length and bcrypt byte limit), StudentBase (name, student ID pattern, e-mail format), StudentSkillCreate (skill ID and two flags), RequestCreate (recipient, skill, title 2–160 and message up to 2000 characters), RequestStatusUpdate (status limited to accepted, rejected or completed), CourseCreate (lessons at least 1), ProgressUpdate (non-negative lessons), SessionCreate (duration 1–1440 minutes) and ReviewCreate (rating 1–5, text 2–2000 characters).

Error handling and response conventions

All responses use standard HTTP status codes: 200 and 201 for success, 204 for a successful deletion, 400 for a logically invalid request (for example a request to oneself), 401 for a missing or invalid token or wrong credentials, 403 for a forbidden action, 404 for a missing resource, 409 for a conflict (duplicate or invalid state change) and 422 for validation errors. Error responses carry a detail message that the front end shows to the user. When the server cannot be reached, the front end displays a message that the back end is unavailable.

6.4 Authentication & Authorization

Authentication establishes who the user is. Passwords are never stored in clear text: passlib's CryptContext with the bcrypt scheme hashes the password at registration and verifies it at login. After a successful login the server creates a JSON Web Token with python-jose, signed with the HS256 algorithm and the secret SECRET_KEY read from the environment. The token carries the student's ID in the sub claim and an expiry time; the lifetime is taken from ACCESS_TOKEN_EXPIRE_MINUTES (default 120 minutes). The front end stores the token in localStorage and sends it in the Authorization header of every request.

Token validation. The dependency get_current_student (based on OAuth2PasswordBearer) decodes the token, checks the signature and expiry, loads the student from the database and rejects the request with 401 if anything is wrong. When the application starts in the browser

and a token is stored, it calls GET /auth/me; if the token is rejected the stored token is removed and the user is signed out. Logging out removes the token and the stored user from the browser.

Authorisation decides what an authenticated user may do. Three mechanisms are used: (1) *Ownership checks* — a student can read or change only his or her own profile and skills and can see only the requests, sessions, enrollments and reviews that belong to him or her; (2) *Participant checks* — only the recipient can accept or reject a request, only participants can complete it, and only participants of a completed exchange can review it; (3) *Role check* — the dependency require_admin returns 403 unless the student's is_admin flag is true. On the client the components ProtectedRoute and AdminRoute redirect users who are not signed in or are not administrators; this is a convenience, and the real enforcement is on the server.

Additional protections. CORS allows only the configured front-end origins; Pydantic validation rejects malformed input; SQLAlchemy builds parameterised queries, which protects against SQL injection; secrets are read from environment variables and the .env file is excluded from version control; the registration schema ignores any attempt to supply an administrator flag. Limitations of this design, such as storing the token in localStorage and the absence of token revocation, are discussed in Section 10.3.

CHAPTER - 7 FEATURES

7.1 List of Main Features

The Campus Skill Exchange & Micro-Learning Platform provides the following main features. Each feature is described in detail in Section 7.2.

No.	Feature	Brief description
1	Registration, login and JWT authentication	Secure student accounts with bcrypt-hashed passwords and signed access tokens; protected routes.
2	Student profile	View and edit name, e-mail, course, year and bio; summary of learning hours and rating.
3	Skill library and skill assignment	Browse and search skills; add or remove teaching and learning skills.
4	Complementary skill matching	Reciprocal and one-way matches computed from stored skills; own account excluded.
5	Learning requests	Send, view, accept, reject and complete peer-learning requests with status tracking.
6	Course catalogue and enrollment	Browse and filter micro-learning courses; duplicate-safe enrollment.
7	Progress tracking	Lesson-based progress per course and overall progress.
8	Learning sessions	Log sessions with duration, date and notes; cumulative learning hours.
9	Reviews and ratings	1–5 star reviews after a completed exchange; average rating summaries.
10	Dashboard	Overview of skills, suggested people, recent requests, progress and key figures.
11	Administrator module	Platform overview, all requests, student list, skill creation, course creation and editing.
12	Public home page and statistics	Database-backed totals and featured skills and courses for visitors.

7.2 How Each Feature Works (User Perspective + Technical Description)

1. Registration, Login and JWT Authentication

User perspective: A visitor opens the Register page, enters name, student ID, e-mail, course, year and a password with confirmation, and submits. After registration the user logs in with the student ID or e-mail. Protected pages open only after login; after logging out they redirect to the login page.

Technical description: The server validates the input, checks that the e-mail and student ID are unique, hashes the password with bcrypt and stores the student. On login it verifies the hash and returns a signed JWT (HS256, default lifetime 120 minutes). The client stores it and sends

it as a Bearer token; the dependency get_current_student validates it on every protected endpoint.

2. Student Profile

User perspective: The student sees his or her name, course and year, edits the details and bio, and sees learning hours, completed courses, average rating and number of reviews.

Technical description: GET /students/me/summary aggregates data from student_skills, enrollments, sessions and reviews with SQL count, sum and avg functions; PUT /students/{id} updates the fields supplied. Access is limited to the owner or an administrator.

3. Skill Library and Skill Assignment

User perspective: Anyone can browse the library, search and filter by category. A signed-in student selects Teach or Learn and presses a skill card to add or remove it; the skills appear as tags in the student's profile section.

Technical description: GET /skills joins skills with student_skills and groups the rows to return counts of teachers and learners. POST /students/{id}/skills creates or updates the student_skills row (unique per student and skill); DELETE removes it. Unauthorised changes return 403.

4. Complementary Skill Matching

User perspective: The student opens Matching and sees other students who can teach what he or she wants to learn. Reciprocal matches (both can help each other) appear first; the match card offers a button to send a request.

Technical description: The back end loads the student's skills and all other students' skills, intersects the sets in memory and returns the results sorted by score (100 reciprocal, 60 one-way), excluding the logged-in student. Nothing is stored (Section 5.5).

5. Learning Requests

User perspective: The student chooses a learning partner and a skill that the partner teaches, adds a title and a message, and sends the request. The recipient sees it under Incoming and accepts or rejects it. Either participant completes an accepted request. Tabs show the requests by status.

Technical description: POST /requests checks that the recipient teaches the skill and that no active duplicate exists, then inserts a row with status pending. PUT/PATCH /requests/{id} applies the state rules of Section 5.4 and returns 403 or 409 for illegal changes.

6. Course Catalogue and Enrollment

User perspective: The user browses courses, searches, filters by category and level and presses Enroll now. The page confirms the enrollment, and the course appears under My enrollments.

Technical description: GET /courses returns courses with a live enrollment count (outer join and GROUP BY). POST /courses/{id}/enroll creates an enrollments row; if one exists it returns it unchanged, and the UNIQUE constraint protects against simultaneous duplicates.

7. Progress Tracking

User perspective: On the Progress page the student sees the progress of each enrolled course and an overall percentage, and can advance a course by one lesson.

Technical description: PUT /progress/{enrollment_id} sets completed_lessons (not above the number of lessons of the course) and computes progress as the rounded percentage; the page averages the progress of all enrollments for the overall figure.

8. Learning Sessions

User perspective: The student logs a session by choosing a course or skill, the duration in minutes, the date and optional notes. The page shows total learning hours, the number of completed sessions and a list of recent sessions.

Technical description: POST /progress/sessions validates the references and the duration and stores a sessions row with status completed. Learning hours are the sum of completed session minutes divided by 60.

9. Reviews and Ratings

User perspective: After an exchange is completed, each participant can choose it on the Reviews page, give 1 to 5 stars and write a review. The page lists received and given reviews and the average rating.

Technical description: POST /reviews checks that the exchange is completed, that the reviewer took part, that the partner is the other participant and that no review exists for this reviewer and exchange. The average rating is computed with AVG over reviews where the student is the partner.

10. Dashboard

User perspective: After login the student sees a greeting, key figures (skills shared, active courses, learning progress and average rating), the skills he or she teaches and wants to learn, suggested people, recent requests and progress.

Technical description: The Dashboard page requests skills, matches, requests, progress, reviews and the summary in parallel and derives the displayed figures in the browser.

11. Administrator Module

User perspective: An administrator sees totals for students, skills, active requests, courses and reviews, lists of students and requests, a form to add skills, and controls to add or edit courses.

Technical description: The endpoints under /admin and the write endpoints for skills and courses depend on require_admin. The front end hides the Admin link from other users and redirects them if they open /admin directly.

12. Public Home Page and Statistics

User perspective: A visitor sees the number of students, skills, courses and completed exchanges and featured skills and courses before registering.

Technical description: GET /public/stats counts rows in the corresponding tables and returns the most recent skills and the most-enrolled courses; it needs no authentication.

CHAPTER - 8 TESTING

Testing verifies that the platform behaves correctly and securely. Because the system is an API-driven application backed by a relational database, testing concentrated on the REST API, the business rules and the persistence of data. The results in this chapter come from tests that were executed against a running instance of the application on 3 October 2026; tests that were not executed are marked Not Tested. No performance, load or coverage figures are reported because none were measured.

8.1 API Testing (Swagger UI and Scripted Requests)

No Postman collection exists in the repository, so Postman is not claimed as a testing tool. The REST API was tested independently of the user interface in two ways. First, FastAPI's Swagger UI at /docs was confirmed to be available (HTTP 200) and provides interactive documentation of every endpoint. Second, a Python script sent real HTTP requests to the running back end and checked the status code and the content of each response.

For each feature the requests covered the positive case (valid input and a permitted user) and negative cases (invalid input, missing token, another student's data, wrong state and duplicates). Two test accounts were registered through the API for the exchange scenario, and the seeded administrator account was used for the administrator checks.

Backend (API) testing

Request	Scenario tested	Expected status
POST /auth/register	Valid; duplicate; malformed student ID	201; 409; 422
POST /auth/login	Valid; wrong password	200 with token; 401
GET /auth/me	Without token; with token	401; 200
PUT /students/{id}	Own profile; another student's record	200; 403
POST, DELETE /students/{id}/skills	Teach; learn; no direction; remove	201; 201; 422; 204
GET /matches	Complementary students; own account excluded	200 with reciprocal match
POST /requests	Valid; to self; skill not taught; duplicate	201; 400; 422; 409
PATCH /requests/{id}	Requester accepts; recipient accepts; complete	403; 200; 200
POST /reviews	Before completion; eligible; duplicate; self	409; 201; 409; 400
POST /courses/{id}/enroll	First enrollment; repeated enrollment	201; 200 (same enrollment)

Request	Scenario tested	Expected status
POST /progress/sessions, PUT /progress/{id}	Valid session; no reference; progress update	201; 422; 200
GET /admin/overview	Student; administrator	403; 200
OPTIONS /auth/login (CORS)	Configured origin; unknown origin	Origin allowed; not allowed

Frontend testing

The front end was checked with the static-analysis and build commands of the project (npm run lint and npm run build), both of which succeeded. The repository does not contain an automated browser-test suite. Browser-level workflows were recorded manually by the team in the file FINAL_E2E_TEST_REPORT.md of the repository; those workflows were not re-executed while this report was prepared and are therefore listed as Not Tested in Section 8.4.

8.2 Integration Testing

Integration testing checks that the modules work correctly together and that the data is stored correctly in the database. The following end-to-end scenarios were executed through the API against a MySQL-compatible database server (Section 8.3):

- **Exchange scenario.** Two students registered and logged in; each added a teaching and a learning skill; the first student's matches contained the second as a reciprocal match; a learning request was sent, accepted by the recipient and completed; a review was submitted; and the reviewed student's summary showed an average rating of 5.0 with one review.
- **Course scenario.** A student enrolled in a course, enrolled again (the same enrollment was returned), logged a 30-minute session and advanced the course by three lessons, which produced a progress value of 50 %.
- **Authorisation scenario.** Requests without a token, with another student's ID and with a non-administrator account were refused with 401 or 403.

After the scenarios, the database was queried directly to confirm that the data had been stored as intended. The checks below were run with SQL statements in the MySQL command-line client.

Database check	Result
Registered test accounts present in students	2 rows found
Passwords stored as bcrypt hashes (value begins with the bcrypt prefix)	2 of 2 rows; no plain-text password stored

Database check	Result
learning_requests rows with status pending / accepted / completed	Rows present for all three statuses
enrollments row after three lessons of a six-lesson course	completed_lessons = 3, progress = 50
sessions row after logging a session	duration_minutes = 30, status = completed
reviews row after the eligible review	rating = 5 stored
matches table	0 rows; confirms that matching does not write to the table

The following integration scenarios were **not** executed and are not claimed: concurrent requests from several clients, behaviour after a database restart, cross-browser behaviour and responsive-layout checks at different screen sizes.

8.3 Tools Used for Testing

Tool	Purpose
Python script using urllib (api_tests.py)	Sends HTTP requests to the running API and checks status codes and response content.
FastAPI Swagger UI (/docs)	Interactive API documentation; confirmed reachable (HTTP 200).
Python unittest	Runs the repository's unit test of the matching function (backend/tests/test_matches.py).
oxlint (npm run lint)	Static analysis of the front-end source.
Vite (npm run build)	Verifies that the front end compiles into a production bundle.
MySQL command-line client	Confirms that data was persisted and that constraints hold.
Test environment	Ubuntu 24.04, Python 3.12.3, Node.js 22, MariaDB 10.11 (MySQL-compatible) on 127.0.0.1; dependencies as pinned in requirements.txt. MariaDB was used because it was available in the test environment; the team's own runs use MySQL.

8.4 Test Cases and Results

The following test cases define the acceptance criteria exercised against the API. TC = test case; the last column records the outcome of the execution: Pass, Fail or Not Tested. All cases marked Pass were executed on 3 October 2026 and produced the expected result.

A. Authentication and authorisation

ID	Test case	Expected result	Result
TC01	Register with valid details	201 Created, student returned	Pass
TC02	Register with duplicate email/student ID	409 Conflict	Pass
TC03	Register with malformed student ID	422 validation error	Pass
TC04	Login with valid credentials	200 OK with JWT access token	Pass

ID	Test case	Expected result	Result
TC05	Login with invalid password	401 Unauthorized	Pass
TC06	Protected endpoint without token	401 Unauthorized	Pass
TC07	Protected endpoint with valid token	200 OK, own profile	Pass
TC08	Update own profile	200 OK, bio persisted	Pass
TC09	Access another student's private profile	403 Forbidden	Pass

B. Skill management

ID	Test case	Expected result	Result
TC10	Browse skill library (public)	200 OK, skill list	Pass
TC11	Add teaching skill	201 Created	Pass
TC12	Add learning skill	201 Created	Pass
TC13	Add skill with no direction selected	422 Unprocessable	Pass
TC14	Remove skill	204 No Content	Pass

C. Skill matching

ID	Test case	Expected result	Result
TC15	Matching identifies complementary student (reciprocal)	Beta listed as Java teacher, match_type=reciprocal, score 100	Pass
TC16	Logged-in student excluded from own matches	No match with own student id	Pass

D. Learning requests

ID	Test case	Expected result	Result
TC17	Request to self	400 Bad Request	Pass
TC18	Request for skill the recipient does not teach	422 Unprocessable	Pass
TC19	Send learning request	201 Created, status=pending	Pass
TC20	Duplicate active request	409 Conflict	Pass
TC21	View sent request	Request listed for requester	Pass
TC22	View received request	Request listed for recipient	Pass
TC23	Requester tries to accept own request	403 Forbidden	Pass
TC24	Recipient accepts request	200 OK, status=accepted	Pass

E. Reviews and ratings

ID	Test case	Expected result	Result
TC25	Review before exchange is completed	409 Conflict	Pass
TC26	Mark accepted request completed	200 OK, status=completed	Pass

ID	Test case	Expected result	Result
TC27	Submit eligible review	201 Created	Pass
TC28	Duplicate review for same exchange	409 Conflict	Pass
TC29	Review with invalid partner (self)	400 Bad Request	Pass
TC30	Rating summary reflects received review	average_rating=5.0, review_count=1	Pass

F. Courses, enrollment and progress

ID	Test case	Expected result	Result
TC31	Browse course catalogue (public)	200 OK, course list	Pass
TC32	Enroll in course	201 Created	Pass
TC33	Duplicate enrollment	200 OK, same enrollment returned (idempotent)	Pass
TC34	Record learning session	201 Created, status=completed	Pass
TC35	Session without course/skill/request	422 Unprocessable	Pass
TC36	Update course progress	200 OK, progress derived from lessons	Pass

G. Administrator access and CORS

ID	Test case	Expected result	Result
TC37	Non-admin calls admin API	403 Forbidden	Pass
TC38	Admin calls admin API	200 OK, platform totals	Pass
TC39	CORS preflight from the configured front-end origin (127.0.0.1:5173)	200 OK; Access-Control-Allow-Origin echoes the origin	Pass
TC40	Request from an origin that is not configured	No Access-Control-Allow-Origin header returned	Pass

H. Browser-level workflows

These workflows require a browser session. They were recorded manually by the team in the repository's QA report but were not re-executed for this report, so they are marked Not Tested here.

ID	Test case	Expected result	Result
BT01	Register through the form	Account created; redirect to the login page	Not Tested
BT02	Log in through the form and refresh the page	Dashboard shown; session persists after refresh	Not Tested
BT03	Open a protected page after logging out	Redirect to the login page	Not Tested
BT04	Add a skill and refresh	Skill remains on the profile	Not Tested
BT05	Send, accept and complete a request through the interface	Status tabs and counts update correctly	Not Tested

ID	Test case	Expected result	Result
BT06	Open the interface on a narrow screen (below 560 px)	Layout adapts without horizontal scrolling	Not Tested

Static checks and automated unit test

No measured performance targets are reported. The checks below are the quality checks that were executed.

Check	Result
Back-end unit test (matching function: reciprocal, one-way, no self-match)	Pass — 1 test run, 1 passed
Front-end lint (npm run lint)	Pass — 0 warnings, 0 errors
Front-end production build (npm run build)	Pass — build completed
Back-end start-up and health check (GET /)	Pass — API running, database connected
Swagger UI availability (GET /docs)	Pass — HTTP 200
Response-time and load measurements	Not Tested
Automated front-end tests	Not available — none exist in the repository

Defect handling. Defects found during development are documented in the repository (FINAL_E2E_TEST_REPORT.md) with the affected file, the cause, the fix and a regression check; they are summarised in Section 10.1.

CHAPTER - 9 DEPLOYMENT

9.1 Steps to Deploy

The current version is configured for local development and evaluation; deployment therefore means starting the three local components — MySQL, the FastAPI back end and the Vite front end — on one machine. The steps below follow the setup documented in the repository README and were checked in a test environment as noted in Section 8.3. The command to start the Vite development server and the back-end commands were executed in that environment; the Windows PowerShell form of the activation command is taken from the repository documentation.

1. **Prepare the machine.** Install Python 3.12 or newer, Node.js with npm, Git and a MySQL server (MySQL Workbench is optional). Clone the repository with git clone and open it in Visual Studio Code.
2. **Create the database.** Start the MySQL service and run database/schema.sql (for example in MySQL Workbench). The script creates the database campus_skill_exchange and its nine tables. Run it only for a new, empty database; database/migrate.sql is a one-time script for an older version of the schema and is not needed for a new installation.
3. **Configure the back end.** In the backend folder copy .env.example to .env and set DATABASE_URL, a private random SECRET_KEY, ACCESS_TOKEN_EXPIRE_MINUTES and CORS_ORIGINS (Section 9.2). The application stops at start-up with a clear message if DATABASE_URL or SECRET_KEY is missing.
4. **Install the back-end dependencies.** Create a virtual environment (python -m venv .venv), activate it and run python -m pip install -r requirements.txt.
5. **Optionally load demonstration data.** Run python -m seed from the backend folder. The seeder is idempotent: it adds a demonstration administrator, seven students, eight skills and three courses without overwriting existing records.
6. **Start the back end.** Run python -m uvicorn main:app --reload --host 127.0.0.1 --port 8000. Open http://127.0.0.1:8000/ to check that the API reports the database as connected and http://127.0.0.1:8000/docs for Swagger UI.
7. **Configure and start the front end.** In the frontend folder copy .env.example to .env (it sets VITE_API_URL=http://127.0.0.1:8000), run npm install, then npm run dev -- --host 127.0.0.1 --port 5173.

8. **Open the application.** Browse to http://127.0.0.1:5173 and register a student or log in with a seeded account.
9. **Verify.** Run the unit test and the front-end checks (python -m unittest discover -s tests -v with PYTHONPATH set to the backend folder; npm run lint; npm run build) and, if required, the SQL queries in database/verify.sql.

Stopping the application consists of stopping the two terminal processes. To use different ports, change the port in the run command and update VITE_API_URL and CORS_ORIGINS so that they agree.

9.2 Environment Configuration

The application reads all environment-specific values from environment files so that the same code can run on different machines. The back end loads backend/.env with python-dotenv; the front end reads VITE_API_URL from frontend/.env through Vite. Both files are excluded from version control, and only the .env.example templates are committed.

Variable	Purpose	Example / placeholder
DATABASE_URL	SQLAlchemy connection URL for MySQL (required)	mysql+pymysql://<db_user>:<your_mysql_password>@127.0.0.1:3306/campus_s
SECRET_KEY	Secret used to sign JWTs (required); must be private and random	<your_secret_key>
ACCESS_TOKEN_EXPIRE_MINUTES	Lifetime of an access token in minutes (default 120)	120
CORS_ORIGINS	Comma-separated list of allowed front-end origins	http://127.0.0.1:5173,http://localhost:5173
VITE_API_URL (front end)	Base URL of the back end (default http://127.0.0.1:8000)	http://127.0.0.1:8000

Sample backend/.env file (placeholders only)

```
DATABASE_URL=mysql+pymysql://<db_user>:<your_mysql_password>@127.0.0.1:3306/campus_skil
l_exchange
SECRET_KEY=<your_secret_key>
ACCESS_TOKEN_EXPIRE_MINUTES=120
CORS_ORIGINS=http://127.0.0.1:5173,http://localhost:5173
```

Sample frontend/.env file

```
VITE_API_URL=http://127.0.0.1:8000
```

Real passwords and keys must never be written in the report, in the repository or in screenshots. A suitable SECRET_KEY can be generated with a password manager or a command such as python -c "import secrets; print(secrets.token_hex(32))".

9.3 Hosting of Frontend and Backend

In the verified current implementation both components are hosted **locally**. No cloud or public hosting is used.

Frontend hosting

The React application is served on http://127.0.0.1:5173 by the Vite development server (npm run dev). The command npm run build compiles the application into static files in the dist folder; this command was executed successfully, but publishing those files on a web server or static-hosting service has not been done and is not claimed.

Backend hosting

The FastAPI application is served on http://127.0.0.1:8000 by Uvicorn, started from the backend folder. The MySQL server runs on the same machine at port 3306. The interactive API documentation is available at /docs while the server is running.

Scaling and reliability

No scaling, load balancing or backup mechanism is implemented. The service is intended for demonstration and evaluation by a small number of users on one machine.

Security in hosting

Because the servers listen on the loopback address, they are not reachable from other machines. Secrets are kept in untracked .env files, the database user should have access only to the campus_skill_exchange schema, and the demonstration accounts created by the seeder use a test-only password and must not be used outside a local demonstration. The project's QA report states that additional hardening and monitoring would be required before any production deployment.

CHAPTER - 10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

The issues described below are taken from the documented evidence of the project: the defects recorded and fixed in the repository's QA report (FINAL_E2E_TEST_REPORT.md), the README and the SQL migration script. No other development difficulties are claimed.

1. Aligning the application with an existing database schema

The repository contains a migration script (database/migrate.sql) that adds columns to learning_requests, courses and sessions of an earlier schema, and the README states that the ORM columns and foreign keys were checked against the existing MySQL tables. Keeping the SQLAlchemy models, the SQL scripts and the live database consistent required care, and destructive schema operations were avoided.

2. Private profile data readable by any signed-in student

The QA report records that the profile endpoint originally did not restrict access to the owner or an administrator, so one student could read another student's record. The access check was added and verified with a 403 response.

3. Loose input validation

Malformed student IDs and overlong passwords were first accepted. A student-ID pattern and a bcrypt byte-length check were added to the Pydantic schemas.

4. Duplicate active learning requests

The back end first allowed a second active request for the same requester, recipient and skill. A conflict check on pending and accepted requests was added, returning 409 and an alert in the interface.

5. Public statistics on the home page

The QA report records that the home-page counts did not reflect the live data; the public statistics endpoint was changed to calculate its counts from the database tables.

6. Error and validation states on the Progress page

Session logging did not consistently validate input and display errors. Validation, loading and error handling were added so that sessions persist and refresh correctly.

7. Profile updates not persisting reliably

Profile fields were not synchronised with the server data and gave no save feedback. The page now loads the server data, updates it and refreshes after saving.

8. Inconsistent administrator interface and API

The administrator route and the administrator data were not consistently enforced. An administrator-only route guard and protected endpoints were added.

9. Legacy mock data conflicting with the real API

Earlier local-storage fixtures in the front end did not match the back end and obscured the real data source. They were removed so that the application always uses the API.

10. Port conflicts during local runs

The README advises choosing a free port when port 8000 is occupied and keeping VITE_API_URL and CORS_ORIGINS consistent; the QA run used alternative ports 8001 and 5174.

10.2 Solutions Applied

Challenge	Solution applied
Schema consistency	Models checked against the live tables; additive migration script; schema.sql as reference; no destructive operations.
Unauthorised profile access	Owner-or-administrator check in the students router (403).
Loose validation	Pydantic Field constraints: student-ID pattern, password length and 72-byte limit.
Duplicate requests	Conflict check on pending and accepted requests for the same requester, recipient and skill (409).
Stale public counts	Statistics computed from the database tables on each call.
Progress page errors	Client-side validation plus loading and error states; data reloaded after each save.
Profile persistence	Form populated from server data and refreshed after saving.
Administrator consistency	require_admin dependency on the API and AdminRoute on the client.
Mock data	Local-storage fixtures removed; all pages use services/api.js.
Port conflicts	Configurable ports with matching VITE_API_URL and CORS_ORIGINS.

10.3 Current Limitations or Known Bugs

Although the platform meets its stated objectives, the following limitations remain in the present version. They are described as limitations of the current scope, not as defects of the implemented functions.

Limitations

- **Local deployment only.** The system is configured for local development and evaluation; there is no cloud hosting, container configuration or automated CI/CD.
- **No notifications or chat.** Students see new requests and reviews only when they open the relevant page; there is no real-time channel, e-mail or push notification, and no messaging between matched students.
- **Simple matching.** Matching compares exact skill entries and uses two fixed scores (100 and 60); it does not weigh proficiency, availability, course or year, and it loads all skill rows into memory for each request.
- **Unused matches table.** The table matches exists in the schema but is not used by the application.
- **No request cancellation or course unenrollment.** The status ENUM has no cancelled value and the interface offers no unenroll action.
- **Limited scheduling.** A session records an optional date and duration entered by the student; there is no calendar, reminder or shared scheduling between the two participants, and each student logs his or her own sessions.
- **Course rating not calculated.** The rating column of courses exists but no code updates it; reviews apply to learning partners, not to courses.
- **Limited administration.** There are no deletion or moderation tools for students, skills or courses in the interface; a course-delete endpoint exists only in the API. Administrator accounts are created outside the application (for example by the seeder).
- **Authentication scope.** Access tokens are stored in the browser's localStorage, there are no refresh tokens or server-side revocation, the minimum password length is six characters, and e-mail ownership is not verified.
- **Schema management.** Tables are created with create_all and SQL scripts; a migration tool such as Alembic is not used.
- **Testing scope.** Only one automated unit test exists; there is no automated front-end or browser test suite, and no load or performance testing was carried out.
- **Web only.** There is no native mobile application.

Known bugs and cosmetic issues

- The avatar in the navigation bar shows the fixed initials “AS” for every user instead of the student's own initials.
- The dashboard greeting always reads “Good morning”, regardless of the time of day.

- The sessions status default differs between the SQL script (completed) and the ORM class (scheduled); the application always sets the status explicitly, so the difference has no effect in normal use.

CHAPTER - 11 FUTURE ENHANCEMENTS

11.1 Planned Features

The architecture is modular, so new capabilities can be added without redesigning the system. The enhancements below are future work; none of them is part of the current implementation.

- **Advanced recommendation algorithm.** Weighted matching that considers proficiency levels, course and year, availability and past review ratings, ranking candidates by a richer score.
- **Real-time notifications.** In-app, e-mail or push notifications when a request is received, accepted or completed.
- **Chat between matched students.** A messaging channel attached to an accepted request.
- **Calendar-based session scheduling.** Proposing and confirming session times between the two participants, with reminders.
- **Richer search and filtering.** Search by teacher rating, availability, course and year.
- **Advanced analytics.** Dashboards for learners and administrators showing learning hours, popular skills and course completion.
- **Badges and gamification.** Recognition for completed exchanges, learning streaks and helpful reviews.
- **Request cancellation and course unenrollment.** Extending the status model and the interface.
- **Course ratings.** Allowing students to rate courses and calculating the stored rating from those ratings.
- **Mobile application.** A native or cross-platform app that uses the same REST API.

11.2 Possible Integrations or Optimizations

Performance and scalability

- Replace the in-memory matching with an SQL join or a materialised matches table (the existing matches table could serve this purpose) and add pagination to long lists.
- Introduce a migration tool such as Alembic for controlled schema changes.
- Cloud deployment of the database and API, a production web server and HTTPS, with automated CI/CD for testing and releases.
- Caching of public statistics and catalogue endpoints.

Integrations

- Institutional single sign-on instead of separate passwords, and e-mail verification of student accounts.
- E-mail or SMS providers for delivery of notifications.
- AI-assisted learning recommendations that suggest skills and courses based on a student's profile and activity.
- Observability tools for logging and error tracking in a production environment.

Security and quality

- Refresh tokens, token revocation and safer token storage (for example HTTP-only cookies).
- A stronger password policy and rate limiting on login.
- Automated back-end integration tests and browser tests (for example with pytest and Playwright) to cover the manual test workflows.

Usability

- Progressive Web App features for installable access on phones.
- Accessibility improvements such as keyboard shortcuts and screen-reader announcements for status changes.

CHAPTER - 12 CONCLUSION

12.1 Summary of the Project

The Campus Skill Exchange & Micro-Learning Platform is a web application that connects students who can teach a skill with students who want to learn it, and that records the resulting learning activity. Students register, describe the skills they can teach and want to learn, find complementary peers, exchange structured learning requests, follow micro-learning courses, log learning sessions and review each other after a completed exchange.

The system is built as a layered web application. A React front end built with Vite presents public, student and administrator pages and talks to a FastAPI back end through a REST API. The back end validates requests with Pydantic, authenticates users with bcrypt-hashed passwords and signed JWT access tokens, and stores all data in a MySQL database through the SQLAlchemy ORM. Ownership, participant and role checks protect private data.

The technical core of the project is the combination of a carefully constrained relational schema and a small set of business rules: the many-to-many skill assignment with a teach/learn direction, the dynamically computed reciprocal and one-way matching, the request life-cycle with enforced transitions, duplicate-safe enrollment, and review eligibility that depends on a completed exchange.

This documentation presents the requirements, technology stack, architecture, design models (use cases, ER diagram, data dictionary, life-cycle and matching algorithm), implementation, features, testing, deployment, limitations and future scope of the platform. The statements in it were checked against the source code, and the results reported in the testing chapter come from tests that were executed against a running instance.

12.2 What Was Achieved

The project achieved its objectives as follows:

Objective	Outcome
Relational database design	Nine-table MySQL schema with primary keys, foreign keys, unique and check constraints and indexes.
Secure authentication	Registration and login with bcrypt hashing, JWT access tokens, protected routes and ownership and role checks.
Skill library and assignments	Searchable library; students record teaching and learning skills in a junction table.
Skill matching	Reciprocal and one-way matches computed dynamically; own account excluded.

Objective	Outcome
Learning-request workflow	Four-state life-cycle with enforced transitions and duplicate-request prevention.
Micro-learning courses	Course catalogue with filters and duplicate-safe enrollment.
Progress and sessions	Lesson-based progress and logged sessions with cumulative learning hours.
Reviews and ratings	Reviews allowed only for completed exchanges, one per reviewer, with average ratings.
Administration	Administrator overview, request and student lists, skill creation and course management (partial).
Testing and documentation	Forty API test cases executed and passed; unit test, lint and build passed; complete documentation produced.

In addition, the project produced a documented REST API (Swagger UI), SQL scripts, a demonstration-data seeder and testing documents that other students can use to run and examine the system.

12.3 Skills Learned During Development

Technical skills

- **Python web development** — building REST APIs with FastAPI, Pydantic validation and dependency injection.
- **Database engineering** — ER modelling, junction tables, constraints, indexes, aggregate SQL and transactions in MySQL.
- **ORM usage** — mapping tables to classes and writing queries with SQLAlchemy and PyMySQL.
- **Security basics** — password hashing with bcrypt, JWT tokens, role-based access and input validation.
- **React development** — components, hooks, Context API, React Router, protected routes and a shared API client.
- **Testing** — designing positive and negative API test cases, verifying persistence directly in the database and writing a unit test.
- **Version control** — Git and GitHub, environment configuration and keeping secrets out of the repository.

Professional and soft skills

- Dividing a project into modules and agreeing interfaces between the front end, the back end and the database.

- Planning and presenting progress in reviews to the guide.
- Technical writing and documentation, including diagrams and data dictionaries.
- Reasoning about trade-offs such as simplicity versus scalability and in-memory matching versus database joins.
- Teamwork, communication and time management under a project schedule.

In conclusion, the Campus Skill Exchange & Micro-Learning Platform shows that a well-designed relational schema, a clean REST API and token-based security can be combined into a practical tool that makes peer learning on a campus visible, organised and measurable.

CHAPTER - 13 REFERENCES

Books

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson Education.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.

Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

Banks, A., & Porcello, E. (2020). Learning React (2nd ed.). O'Reilly Media.

Standards and Specifications

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT), RFC 7519. Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519

Jones, M., & Hardt, D. (2012). The OAuth 2.0 Authorization Framework: Bearer Token Usage, RFC 6750. Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc6750

Fielding, R., Nottingham, M., & Reschke, J. (2022). HTTP Semantics, RFC 9110. Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc9110

OWASP Foundation. OWASP Top Ten Web Application Security Risks. https://owasp.org/www-project-top-ten/

Official Documentation

Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/

Ramírez, S. FastAPI Documentation. https://fastapi.tiangolo.com/

Pydantic. Pydantic Documentation. https://docs.pydantic.dev/

SQLAlchemy. SQLAlchemy 2.0 Documentation. https://docs.sqlalchemy.org/en/20/

Oracle Corporation. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

PyMySQL. PyMySQL Documentation. https://pymysql.readthedocs.io/

python-jose. python-jose Documentation. https://python-jose.readthedocs.io/

Passlib. Passlib Documentation. https://passlib.readthedocs.io/

Uvicorn. Uvicorn Documentation. https://www.uvicorn.org/

React. React Documentation. https://react.dev

React Router. React Router Documentation. https://reactrouter.com

Vite. Vite Documentation. https://vite.dev

Git. Git Documentation. https://git-scm.com/doc

GitHub, Inc. GitHub Docs. https://docs.github.com

Microsoft. Visual Studio Code Documentation. https://code.visualstudio.com/docs

Mozilla. MDN Web Docs (HTML, CSS, JavaScript, Fetch API). https://developer.mozilla.org

Tutorials and Learning Resources

FastAPI. Tutorial – User Guide. https://fastapi.tiangolo.com/tutorial/

React. Learn React. https://react.dev/learn

Project Resources

Ggayathri1730. DBMS-Campus-Skill-Exchange-Micro-Learning-Platform (source code, SQL scripts and project documents). https://github.com/Ggayathri1730/DBMS-Campus-Skill-Exchange-Micro-Learning-Platform

Course Material

Koneru Lakshmaiah Education Foundation. Database Systems Engineering and Distributed Backend Development (25CS1302E) — course notes and project guidelines, academic year 2026–2027.

CHAPTER - 14 APPENDICES

A. Screenshots of the Application

The figures below are screenshots of the running application captured from the final working build. The typed entries in the registration and login forms have been blurred. A screenshot of the Matching page and one of the Administrator page were not supplied and are marked as required.

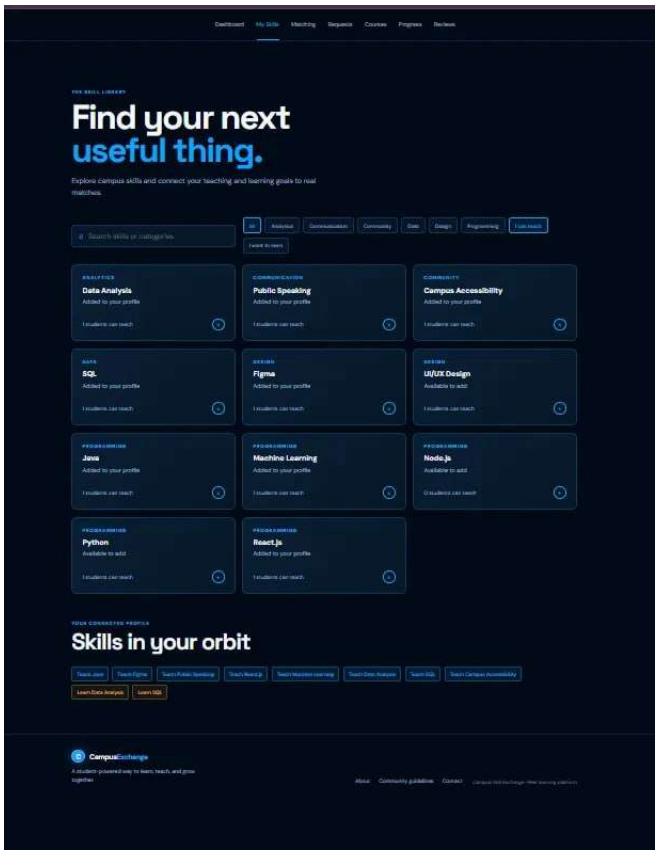


Figure A1: My Skills page — skill library with teaching and learning skills



Figure A3: Student dashboard (skills, requests, progress)

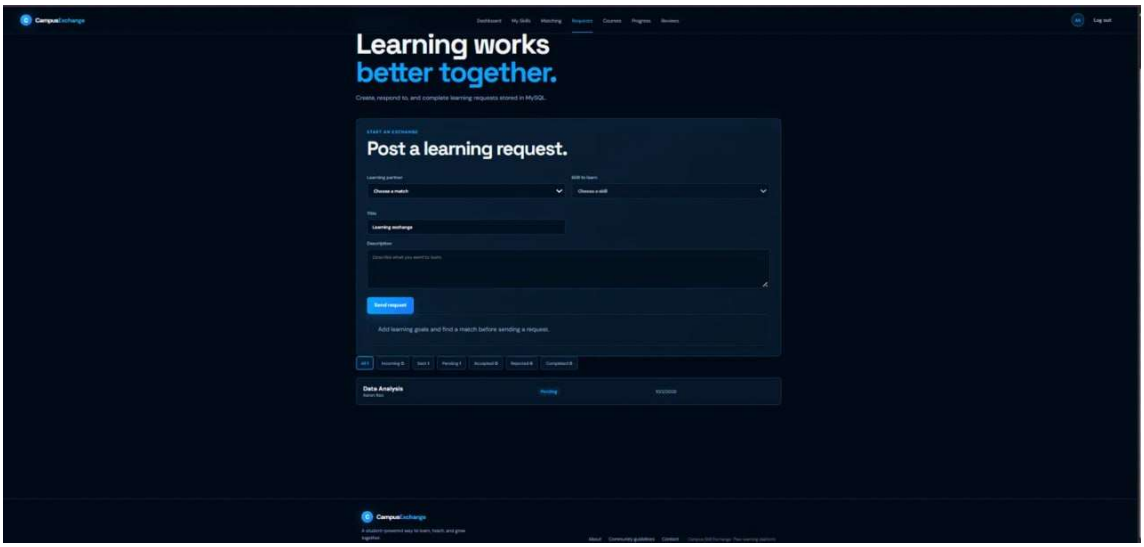


Figure A4: Learning requests page (post a request, status tabs, request list)

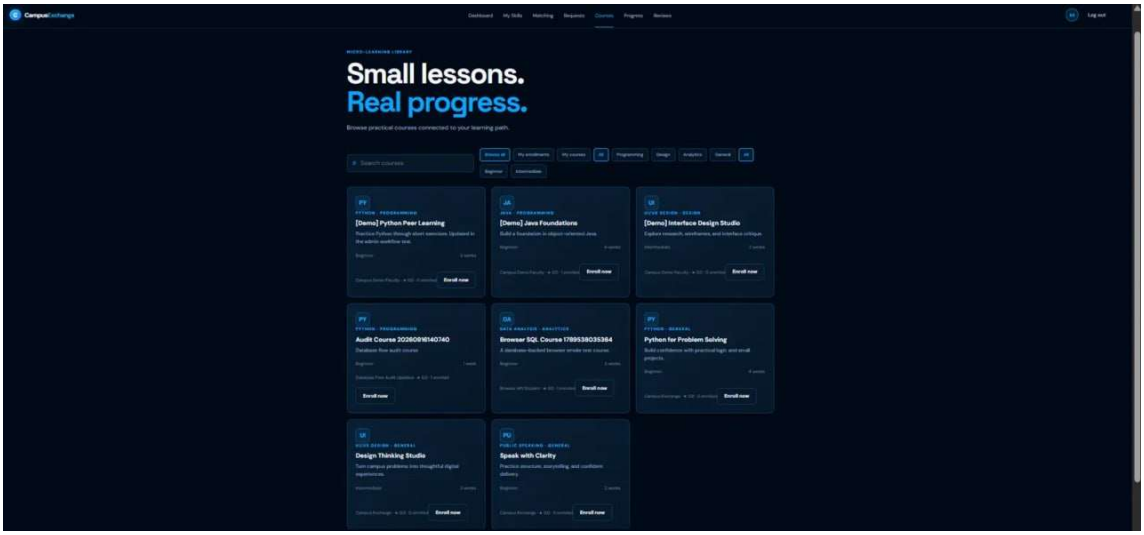


Figure A5: Micro-learning course catalogue with filters and enrollment

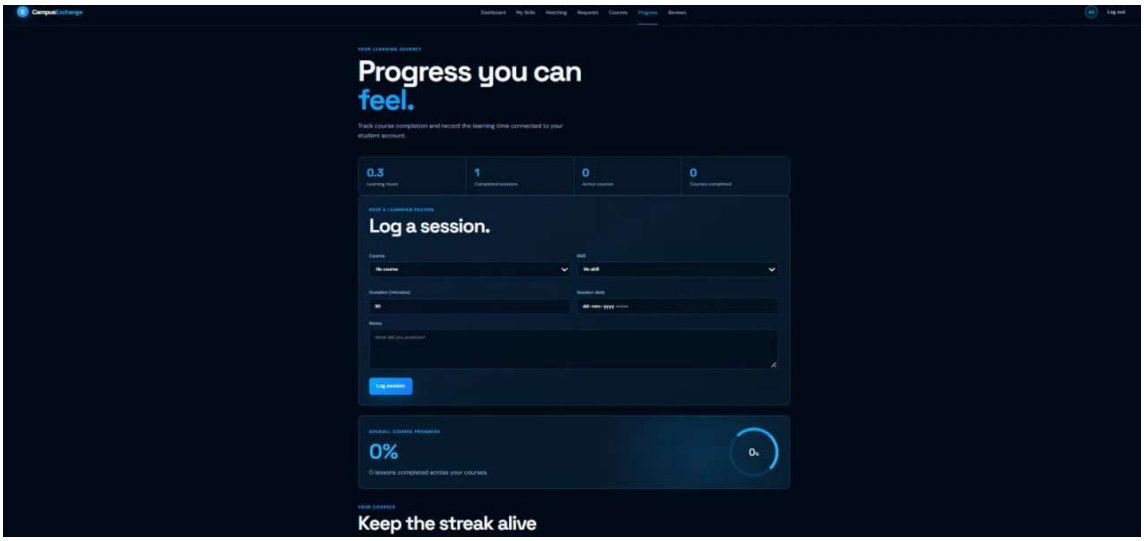


Figure A6: Progress page (learning hours, log a session, overall progress)

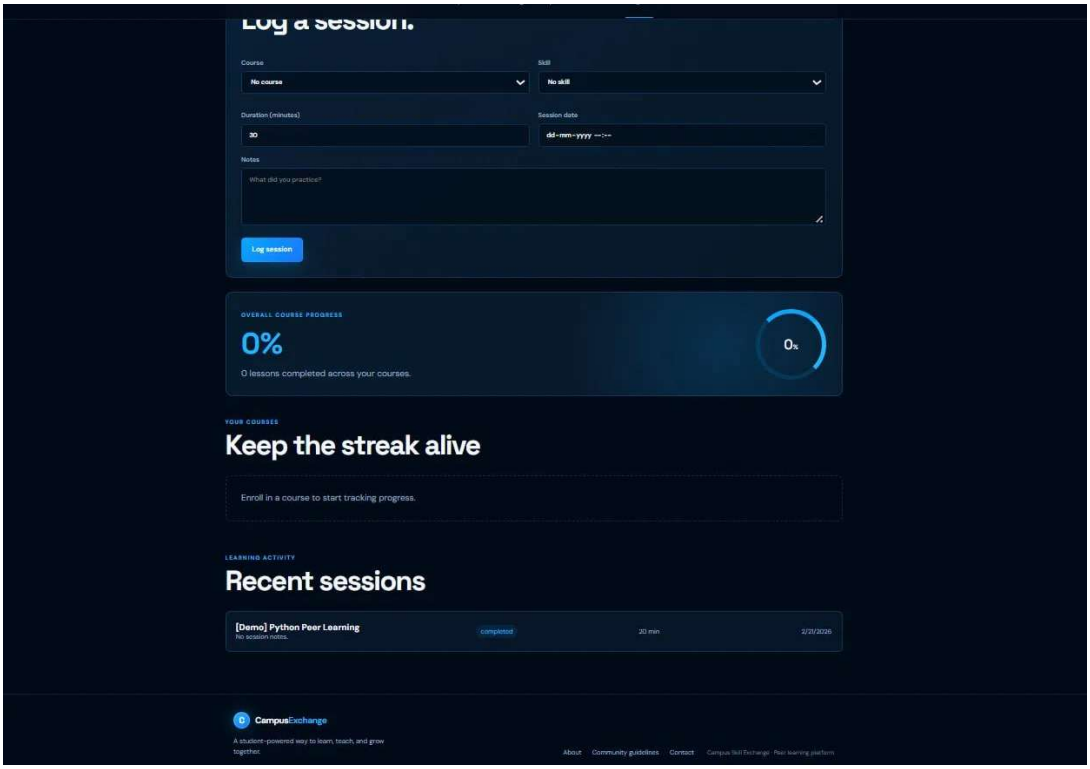


Figure A7: Progress page — courses and recent sessions



Figure A8: Reviews page (received and given reviews, add a review)

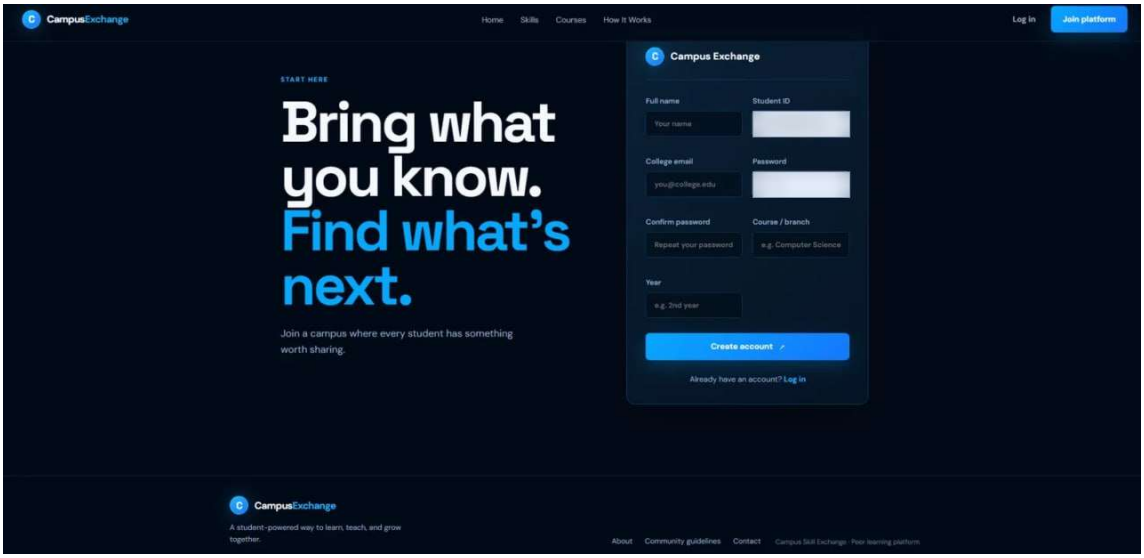


Figure A9: Registration page (Join platform)

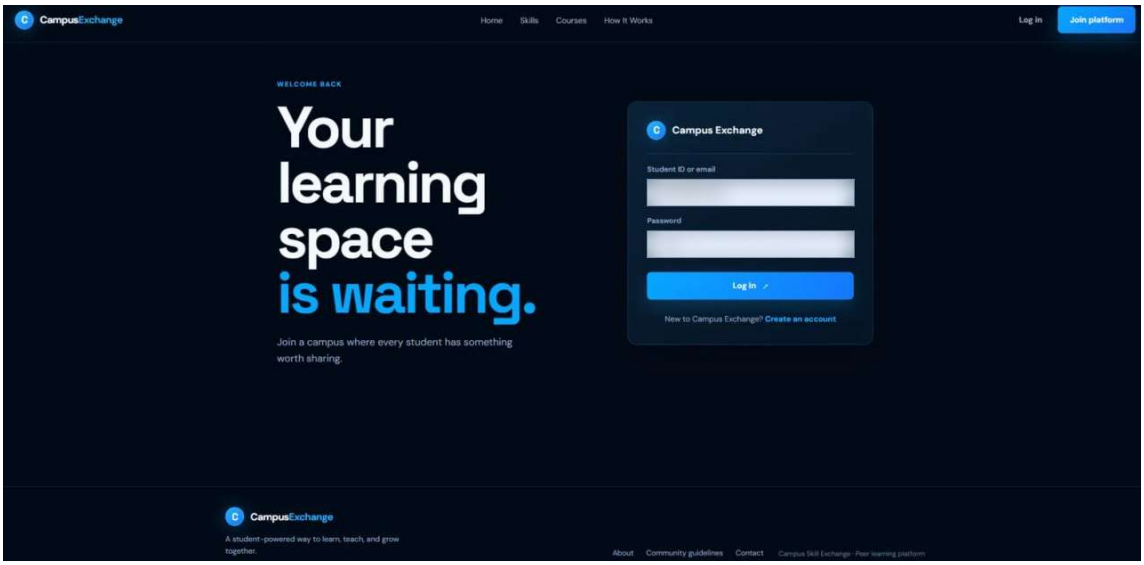


Figure A10: Login page

B. Sample Code Snippets

The following excerpts are taken unchanged from the repository and show the most important parts of the implementation: the database schema, the ORM models, JWT authentication, the matching algorithm, the learning-request endpoints, enrollment, progress and sessions, review rules and the React authentication code. File paths are given above each snippet. Secrets are not included.

B.1 MySQL schema (student_skills and learning_requests)

File: `database/schema.sql`. The junction table with its unique and check constraints, and the learning-request table with the status ENUM.

```
CREATE TABLE IF NOT EXISTS student_skills (
  id INT AUTO_INCREMENT PRIMARY KEY,
  student_id INT NOT NULL,
  skill_id INT NOT NULL,
  can_teach BOOLEAN NOT NULL DEFAULT FALSE,
  wants_to_learn BOOLEAN NOT NULL DEFAULT FALSE,
  CONSTRAINT uq_student_skill UNIQUE (student_id, skill_id),
  CONSTRAINT ck_skill_direction CHECK (can_teach = TRUE OR wants_to_learn = TRUE),
  CONSTRAINT fk_student_skills_student FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE
CASCADE,
  CONSTRAINT fk_student_skills_skill FOREIGN KEY (skill_id) REFERENCES skills(id) ON DELETE
CASCADE,
  INDEX ix_student_skills_student (student_id),
  INDEX ix_student_skills_skill (skill_id)
) ENGINE=InnoDB;


CREATE TABLE IF NOT EXISTS learning_requests (
  id INT AUTO_INCREMENT PRIMARY KEY,
  requester_id INT NOT NULL,
  recipient_id INT NOT NULL,
  skill_id INT NOT NULL,
  title VARCHAR(160) NOT NULL DEFAULT 'Learning exchange',
  status ENUM('pending', 'accepted', 'rejected', 'completed') NOT NULL DEFAULT 'pending',
  message TEXT NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
```

```
  CONSTRAINT fk_requests_requester FOREIGN KEY (requester_id) REFERENCES students(id) ON DELETE
CASCADE,
  CONSTRAINT fk_requests_recipient FOREIGN KEY (recipient_id) REFERENCES students(id) ON DELETE
CASCADE,
  CONSTRAINT fk_requests_skill FOREIGN KEY (skill_id) REFERENCES skills(id),
  CONSTRAINT ck_request_students CHECK (requester_id <> recipient_id),
  INDEX ix_requests_status (status),
  INDEX ix_requests_people (requester_id, recipient_id)
) ENGINE=InnoDB;
```

B.2 SQLAlchemy models (StudentSkill and LearningRequest)

File: `backend/models.py`. ORM classes mapped to the tables above.

```
class StudentSkill(Base):
    __tablename__ = "student_skills"
    __table_args__ = (UniqueConstraint("student_id", "skill_id", name="uq_student_skill"),)


    id: Mapped[int] = mapped_column(primary_key=True)
    student_id: Mapped[int] = mapped_column(ForeignKey("students.id", ondelete="CASCADE"),
index=True, nullable=False)
    skill_id: Mapped[int] = mapped_column(ForeignKey("skills.id", ondelete="CASCADE"), index=True,
nullable=False)
    can_teach: Mapped[bool] = mapped_column(Boolean, default=False, nullable=False)
    wants_to_learn: Mapped[bool] = mapped_column(Boolean, default=False, nullable=False)


    student = relationship("Student", back_populates="skills")
    skill = relationship("Skill")


class LearningRequest(Base):
    __tablename__ = "learning_requests"


    id: Mapped[int] = mapped_column(primary_key=True)
    requester_id: Mapped[int] = mapped_column(ForeignKey("students.id", ondelete="CASCADE"),
index=True, nullable=False)
    recipient_id: Mapped[int] = mapped_column(ForeignKey("students.id", ondelete="CASCADE"),
index=True, nullable=False)
    skill_id: Mapped[int] = mapped_column(ForeignKey("skills.id"), nullable=False)
    title: Mapped[str] = mapped_column(String(160), default="Learning exchange", nullable=False)
    status: Mapped[str] = mapped_column(String(20), default="pending", index=True, nullable=False)
    message: Mapped[str] = mapped_column(Text, default="", nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, server_default=func.now(),
nullable=False)
    updated_at: Mapped[datetime] = mapped_column(DateTime, server_default=func.now(),
onupdate=func.now(), nullable=False)


    requester = relationship("Student", foreign_keys=[requester_id])
    recipient = relationship("Student", foreign_keys=[recipient_id])
    skill = relationship("Skill")
```

B.3 Password hashing and JWT authentication

File: `backend/auth.py`. Token settings, bcrypt hashing, token creation, the current-student dependency and the administrator guard.

```
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES", "120"))
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")



def hash_password(password: str) -> str:
    return pwd_context.hash(password)



def verify_password(password: str, password_hash: str) -> bool:
    return pwd_context.verify(password, password_hash)



def create_access_token(student_id: int) -> str:
    expires = datetime.now(timezone.utc) + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
```

```
    return jwt.encode({"sub": str(student_id), "exp": expires}, SECRET_KEY, algorithm=ALGORITHM)



def get_current_student(token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)) ->
Student:
    credentials_error = HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid or
expired authentication token", headers={"WWW-Authenticate": "Bearer"})
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        student_id = int(payload.get("sub", ""))
    except (JWTError, ValueError, TypeError):
        raise credentials_error
    student = db.get(Student, student_id)
    if student is None:
        raise credentials_error
    return student



def require_admin(student: Student = Depends(get_current_student)) -> Student:
    if not student.is_admin:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Admin access required")
    return student
```

B.4 Skill matching algorithm

File: `backend/routers/matches.py`. Computes reciprocal and one-way matches from the skill rows and excludes the current student.

```
def build_match_results(student_id: int, own_skills: list[dict], other_skills: list[dict]) ->
list[dict]:
    own_teach = {item["skill_id"]: item["skill_name"] for item in own_skills if item["can_teach"]}
    own_learn = {item["skill_id"]: item["skill_name"] for item in own_skills if
item["wants_to_learn"]}
    candidates = defaultdict(list)
    for item in other_skills:
        if item["student_id"] != student_id:
            candidates[item["student_id"]].append(item)


    matches = []
    for candidate in candidates.values():
        profile = candidate[0]
        they_teach = {item["skill_id"]: item["skill_name"] for item in candidate if
item["can_teach"]}
        they_learn = {item["skill_id"]: item["skill_name"] for item in candidate if
item["wants_to_learn"]}
        desired_skills = own_learn.keys() & they_teach.keys()
        reciprocal_skills = sorted(own_teach.keys() & they_learn.keys())
        for skill_id in sorted(desired_skills):
            is_reciprocal = bool(reciprocal_skills)
            matches.append({
                "student_id": profile["student_id"],
                "student_name": profile["student_name"],
                "course": f"{profile['course']}, {profile['year']}",
                "skill_id": skill_id,
                "teaches": they_teach[skill_id],
                "wants": ", ".join(they_learn[match_id] for match_id in reciprocal_skills),
                "score": 100 if is_reciprocal else 60,
                "match_type": "reciprocal" if is_reciprocal else "one_way",
                "reciprocal_skills": [own_teach[match_id] for match_id in reciprocal_skills],
            })
    return sorted(matches, key=lambda item: (-item["score"], item["student_name"].casefold(),
item["skill_id"]))
```

B.5 Learning request endpoints

File: `backend/routers/requests.py`. Creating a request with its validation rules, and the status-transition rules of the update endpoint.

```
@router.post("", response_model=RequestResponse, status_code=status.HTTP_201_CREATED)
def create_request(payload: RequestCreate, current: Student = Depends(get_current_student), db:
Session = Depends(get_db)):
    if payload.recipient_id == current.id:
        raise HTTPException(status_code=400, detail="You cannot request a session with yourself")
    recipient = db.get(Student, payload.recipient_id)
    if recipient is None or db.get(Skill, payload.skill_id) is None:
        raise HTTPException(status_code=404, detail="Recipient or skill not found")
    teaches_skill = db.query(StudentSkill).filter_by(student_id=recipient.id,
skill_id=payload.skill_id, can_teach=True).first()
    if teaches_skill is None:
        raise HTTPException(status_code=422, detail="The recipient is not listed as teaching this
skill")
    duplicate = db.query(LearningRequest).filter_by(requester_id=current.id,
recipient_id=recipient.id,
skill_id=payload.skill_id).filter(LearningRequest.status.in_(["pending", "accepted"])).first()
    if duplicate is not None:
        raise HTTPException(status_code=409, detail="An active request for this skill already
exists")
    item = LearningRequest(requester_id=current.id, **payload.model_dump())
    db.add(item)
    try:
        db.commit()
        db.refresh(item)
    except Exception:
        db.rollback()
        raise
    return serialize_request(item)



@router.api_route("/{request_id}", methods=["PUT", "PATCH"], response_model=RequestResponse)
def update_request(request_id: int, payload: RequestStatusUpdate, current: Student =
Depends(get_current_student), db: Session = Depends(get_db)):
    item = db.get(LearningRequest, request_id)
    if item is None:
        raise HTTPException(status_code=404, detail="Learning request not found")
    if current.id not in (item.requester_id, item.recipient_id):
        raise HTTPException(status_code=403, detail="You cannot update this request")
    if item.status == "pending" and payload.status in {"accepted", "rejected"} and current.id !=
item.recipient_id:
        raise HTTPException(status_code=403, detail="Only the recipient can accept or reject a
request")
    if item.status == "accepted" and payload.status == "completed":
        item.status = payload.status
    elif item.status == "pending" and payload.status in {"accepted", "rejected"}:
        item.status = payload.status
    else:
        raise HTTPException(status_code=409, detail=f"Cannot change a {item.status} request to
{payload.status}")
    try:
        db.commit()
        db.refresh(item)
```

B.6 Course enrollment (duplicate-safe)

File: `backend/routers/courses.py`. Returns the existing enrollment if one exists; otherwise creates it; the unique constraint covers simultaneous requests.

```
@router.post("/{course_id}/enroll", response_model=EnrollmentResponse,
status_code=status.HTTP_201_CREATED)
def enroll(course_id: int, response: Response, current: Student = Depends(get_current_student),
db: Session = Depends(get_db)):
    course = db.get(Course, course_id)
    if course is None:
        raise HTTPException(status_code=404, detail="Course not found")
    enrollment = db.query(Enrollment).filter_by(student_id=current.id,
course_id=course_id).first()
    if enrollment is not None:
        response.status_code = status.HTTP_200_OK
        return serialize_enrollment(enrollment)
    enrollment = Enrollment(student_id=current.id, course_id=course_id)
    db.add(enrollment)
    course.enrollment_count += 1
    try:
        db.commit()
        db.refresh(enrollment)
    except IntegrityError:
        db.rollback()
        enrollment = db.query(Enrollment).filter_by(student_id=current.id,
course_id=course_id).first()
```

```
        if enrollment is None:
            raise HTTPException(status_code=409, detail="Enrollment could not be created")
        response.status_code = status.HTTP_200_OK
    return serialize_enrollment(enrollment)
```

B.7 Progress calculation and session logging

File: `backend/routers/progress.py`. Validation of references and of the linked request, and the calculation of course progress from completed lessons.

```
@router.post("/sessions", response_model=SessionResponse, status_code=201)
def create_session(payload: SessionCreate, current: Student = Depends(get_current_student), db:
Session = Depends(get_db)):
    if payload.course_id is None and payload.skill_id is None and payload.request_id is None:
        raise HTTPException(status_code=422, detail="Choose a course, skill, or learning request")
    if payload.course_id is not None and db.get(Course, payload.course_id) is None:
        raise HTTPException(status_code=404, detail="Course not found")
    if payload.skill_id is not None and db.get(Skill, payload.skill_id) is None:
        raise HTTPException(status_code=404, detail="Skill not found")
    if payload.request_id is not None:
        request = db.get(LearningRequest, payload.request_id)
        if request is None:
            raise HTTPException(status_code=404, detail="Learning request not found")
        if current.id not in (request.requester_id, request.recipient_id):
            raise HTTPException(status_code=403, detail="You did not participate in this learning
request")
        if request.status not in {"accepted", "completed"}:
            raise HTTPException(status_code=409, detail="A session requires an accepted learning
request")
    item = LearningSession(student_id=current.id, **payload.model_dump(), status="completed")
    db.add(item)
    try:
        db.commit()
        db.refresh(item)
    except Exception:
        db.rollback()
        raise
    return item



@router.put("/{enrollment_id}", response_model=EnrollmentResponse)
def update_progress(enrollment_id: int, payload: ProgressUpdate, current: Student =
Depends(get_current_student), db: Session = Depends(get_db)):
    item = db.get(Enrollment, enrollment_id)
    if item is None or item.student_id != current.id:
        raise HTTPException(status_code=404, detail="Enrollment not found")
    item.completed_lessons = min(payload.completed_lessons, item.course.lessons)
    item.progress = round(item.completed_lessons / item.course.lessons * 100)
    if payload.quiz_status is not None:
        item.quiz_status = payload.quiz_status
    try:
        db.commit()
        db.refresh(item)
    except Exception:
        db.rollback()
        raise
    return serialize(item)
```

B.8 Review eligibility rules

File: `backend/routers/reviews.py`. A review is accepted only for a completed exchange, from a participant, once per reviewer.

```
@router.post("", response_model=ReviewResponse, status_code=status.HTTP_201_CREATED)
def create_review(payload: ReviewCreate, current: Student = Depends(get_current_student), db:
Session = Depends(get_db)):
    if current.id == payload.partner_id or db.get(Student, payload.partner_id) is None:
        raise HTTPException(status_code=400, detail="A valid learning partner is required")
    exchange = db.get(LearningRequest, payload.request_id)
    if exchange is None:
        raise HTTPException(status_code=404, detail="Learning exchange not found")
    if exchange.status != "completed":
        raise HTTPException(status_code=409, detail="Reviews are available after an exchange is
completed")
    if current.id not in (exchange.requester_id, exchange.recipient_id):
        raise HTTPException(status_code=403, detail="You did not participate in this exchange")
    expected_partner_id = exchange.recipient_id if current.id == exchange.requester_id else
exchange.requester_id
    if payload.partner_id != expected_partner_id:
        raise HTTPException(status_code=422, detail="The selected partner does not match this
exchange")
    duplicate = db.query(Review).filter_by(reviewer_id=current.id, request_id=exchange.id).first()
    if duplicate is not None:
        raise HTTPException(status_code=409, detail="You have already reviewed this exchange")
    review = Review(reviewer_id=current.id, **payload.model_dump())
    db.add(review)
    try:
        db.commit()
    except IntegrityError:
        db.rollback()
        raise HTTPException(status_code=409, detail="A review already exists for this exchange")
    db.refresh(review)
    return serialize(review)
```

B.9 React protected routes

File: `frontend/src/components/ProtectedRoute.jsx`. Redirects visitors who are not signed in, and non-administrators from the administrator page.

```
import { Navigate } from 'react-router-dom'
import { useAuth } from '../context/authContext'


export default function ProtectedRoute({ children }) {
  const { user, loading } = useAuth()
  if (loading) return <div className="loading-state">Loading your learning space...</div>
  if (!user) return <Navigate to="/login" replace />
  return children
}


export function AdminRoute({ children }) {
  const { user, loading } = useAuth()
  if (loading) return <div className="loading-state">Loading your learning space...</div>
  if (!user) return <Navigate to="/login" replace />
  if (!user.is_admin) return <Navigate to="/dashboard" replace />
  return children
}
```

B.10 React API client

File: `frontend/src/services/api.js`. One helper adds the base URL and the Bearer token and converts error responses into messages.

```
const API_URL = import.meta.env.VITE_API_URL || 'http://127.0.0.1:8000'


async function request(path, options = {}) {
  const token = localStorage.getItem('campus_exchange_token')
  const headers = { 'Content-Type': 'application/json', ...(options.headers || {}) }
  if (token) headers.Authorization = `Bearer ${token}`
  let response
  try {
    response = await fetch(`${API_URL}${path}`, { ...options, headers })
  } catch {
    throw new Error(`Backend unavailable at ${API_URL}. Start FastAPI and check backend/.env.`)
  }
  const payload = await response.json().catch(() => ({}))
  if (!response.ok) throw new Error(payload.detail || 'The request could not be completed.')
  return payload
```

```
}
```

C. Installation / Setup Instructions

The project can be run on a development machine running Windows, Linux or macOS. Follow the steps below; Section 9.1 explains each step in more detail.

Prerequisites

- Python 3.12 or newer with pip and venv
- Node.js and npm (front-end tooling)
- MySQL 8.x (and MySQL Workbench or the mysql command-line client)
- Git and Visual Studio Code

Steps

1. Clone the repository: git clone https://github.com/Ggayathri1730/DBMS-Campus-Skill-Exchange-Micro-Learning-Platform and open the folder in VS Code.
2. Create the database: run database/schema.sql in MySQL (it creates campus_skill_exchange and the nine tables).
3. Configure the back end: in the backend folder copy .env.example to .env and set DATABASE_URL=mysql+pymysql://<db_user>:<your_mysql_password>@127.0.0.1:3306/campus_skill_exchange, SECRET_KEY=<your_secret_key>, ACCESS_TOKEN_EXPIRE_MINUTES=120 and CORS_ORIGINS=http://127.0.0.1:5173,http://localhost:5173.
4. Install back-end dependencies: python -m venv .venv, activate it, then python -m pip install -r requirements.txt.
5. Optional demonstration data: python -m seed (from the backend folder).
6. Start the back end: python -m uvicorn main:app --reload --host 127.0.0.1 --port 8000. Open http://127.0.0.1:8000/docs for Swagger UI.
7. Configure and start the front end: in the frontend folder copy .env.example to .env, run npm install and npm run dev -- --host 127.0.0.1 --port 5173.
8. Open http://127.0.0.1:5173, register a student and log in (or use a seeded account).

Common problems

Problem	Solution
The back end stops with “DATABASE_URL is missing” or “SECRET_KEY is missing”	Create backend/.env from .env.example and set both values.

Problem	Solution
The back end cannot connect to MySQL	Check that the MySQL service is running, that the database exists and that the user, password, host and port in DATABASE_URL are correct.
The browser shows “Backend unavailable”	Start FastAPI and check that VITE_API_URL points to the correct host and port.
Requests are blocked by the browser (CORS error)	Add the exact front-end origin, for example http://127.0.0.1:5173, to CORS_ORIGINS and restart the back end.
Port 8000 or 5173 is already in use	Stop only a process you have identified, or choose another port and update VITE_API_URL and CORS_ORIGINS to match.
Registration or login rejects a long password	Passwords must be at most 72 bytes (bcrypt limit) and at least 6 characters.

D. User Manual / Guide

For students

1. Open the CampusExchange website and click Join platform (or Register). Enter your full name, student ID, college e-mail, password, course/branch and year, and press Create account.
2. Log in with your student ID or e-mail and your password. The dashboard opens and shows your skills, suggested people, recent requests and progress.
3. Open Skills (My Skills). Choose I can teach or I want to learn, search or filter by category, and select a skill card to add it to your profile; select it again to remove it.
4. Open Matching to see students who can teach what you want to learn. Reciprocal matches (you can also teach them something) are shown first.
5. To ask for help, send a request from Matching or open Requests, choose a learning partner and a skill, add a title and a message and press Send request.
6. In Requests, use the tabs to see incoming and sent requests. If you receive a request, accept or reject it. When the exchange is finished, either participant marks the accepted request as completed.
7. Open Courses, filter or search, and press Enroll now on a course you like. Your enrolled courses appear under My enrollments.
8. Open Progress to see your learning hours and course progress, advance a course by one lesson and log a learning session (course or skill, duration, date and notes).
9. After an exchange is completed, open Reviews, choose the exchange, give 1–5 stars, write a short review and press Submit review.
10. Use Profile (your avatar) to update your details and Log out when you have finished.

For administrators

1. Log in with an administrator account; the Admin link appears in the navigation bar.
2. Open Admin to view the platform totals, the list of students and all learning requests.
3. Add a new skill to the library with the skill form on the Admin page.
4. Open Courses to create a new course or edit an existing one with the administrator controls.

E. Geo-Tag Photos with Guide

Insert geo-tagged photographs of the team working on the project with the guide (Dr. Purushotham Muniganti), showing date, time and location stamps.

